Introducción a GNU/Linux

LIDSOL
Estudiantes de Ingeniería en Computación

Universidad Nacional Autónoma de México
Facultad de Ingeniería

GNU/Linux para principiantes, Semestre 2026-1

Introducción
- Instalación
  - Creación de una USB Booteble
  - Creación de un espacio libre en disco
  - Pre-Instalación
- Instalación de Manjaro
- Instalación básica de Arch (UEFI)
- GNOME
  - Cajón de aplicaciones
  - Búsqueda
  - QuickSettings
  - Notificaciones
  - Dock
  - Workspace
  - Extensiones
  - Otros entornos de escritorio
- Historia
  - Historia de UNIX
  - GNU

- Linux
- Software libre vs OpenSource
- Distribuciones

El sistema de archivos

- Concepto de archivo
- Concepto de directorio
- Directorios: rutas
- Estructura de directorios
- Concepto de sistema de archivos
- Comandos
  - Comandos para el manejo de directorios
  - Comandos para el manejo de archivos
  - Para directorios y archivos
- Editores de texto
  - vim/neovim
  - nano/micro
- Sistema de Archivos
  - btrfs
  - ext3

- ext4
- vfat
- NTFS
- FAT32

- Almacenamiento
  - Listar dispositivos de almanceamiento (lsblk)
  - Montaje y desmontaje de dispositivos de almacenamiento (mount, umount)
  - Partición de un disco (cfdisk)
  - Creación de un sistema de archivos (mkfs.)
  - Cifrado de un partición (luksFormat)
  - Uso de gnome-disk-utility
  - Respaldos con programa gráfico (pika backup)

Utilidades de Línea de Comandos

- date
- cal
- less
- sort
- uniq

- grep
- awk
- figlet
- fastfetch
- htop

Administración, control del sistema

- Administración de usuarios
  - Creación de usuarios (useradd)
  - Cambiar/agregar la contraseña a un usuario (passwd)
  - Cambiar el nombre de inicio de sesión de un usuario
  - Borrar un usuario
  - Cambiar el directorio personal de un usuario
  - Usuario root
  - Sudo, sudoers
- Grupos
- Permisos
  - Permisos de los archivos (d | rwx | r-x | r-x)
  - Cambiar los permisos de un archivo (chmod)
- Control del sistema

- Reiniciar el sistema
- Apagar el sistema

Red

- ping [ip]
- Listar las interfaces de red
- Activar y desactivar interfaces de red
- Sobre direcciones IP
  - Mostrar direcciones IP
  - Añadir una direccion IP a una interfaz
  - Eliminar una dirección IP de una interfaz
- Gestor de redes
- ssh
  - Conectarse a un host
  - Ejecutar un comando y salir ssh
  - Subir y descargar archivos de un servidor ssh
  - Configuración con clave pública y privada
- Descargar archivos
- curl -O [url]
- wget [url]

Instalación de programas, paquetes y mirrors

- Generalidades de flatpak
- Flathub
- Instalar aplicaciones desde flatpak
- Borrar aplicaciones de flatpak
- Listar aplicaciones de flatpak instaladas
- Gnome Software
- Gestionar permisos de flatpak con interfaz gráfica
- Pacman
  - Instalar paquetes
  - Borrar paquetes
  - Buscar paquetes
  - Actualizar paquetes
  - Mirrors
  - Instalar un programa compilándolo (makepkg -si)
  - AUR Helper (paru)

Intérprete de órdenes

- Entrada y salida, redirección
  - stdout

Procesos y Shell

Shell y Bash scripting

Instalación de GNU/Linux

Instalación de GNU/Linux

Podemos instalar una distribución de GNU/Linux de las siguientes tres maneras:

- Máquina virtual (hosted o bere-metal): Un hipervisor es un software que virtualiza, es decir, crea instancias virtuales de recursos físicos. Se puede instalar una distribución GNU/Linux virtualizada desde Windows con hipervisores como VirtualBox.
- Instalación física con dualboot: Dos sistemas operativos instalados en disco físico. Por ejemplo, GNU/Linux y Windows.
- Instalación física con disco completo: GNU/Linux ocupa todo el disco.

Instalación de GNU/Linux

Para una instalación física, seguiremos los siguientes pasos:

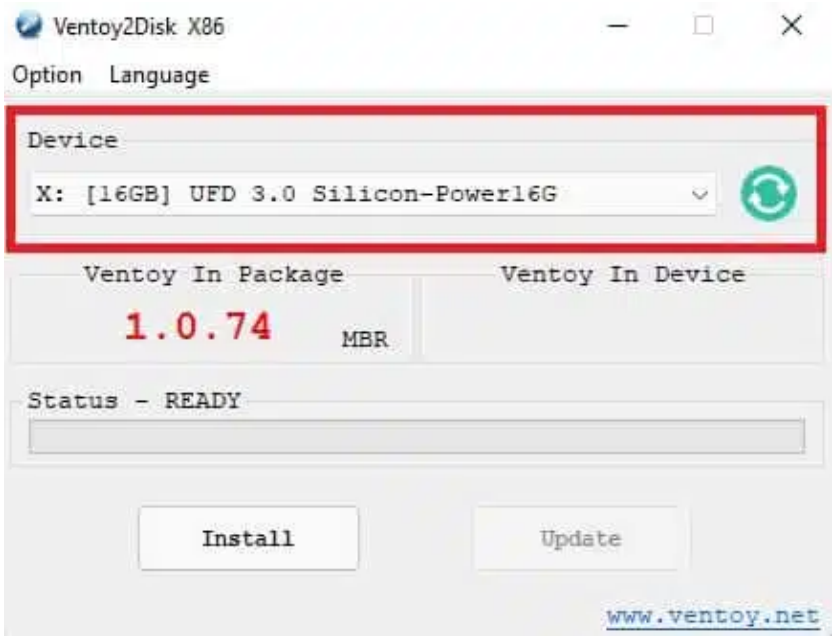
- Creación de una memoria USB booteable (ventoy).
- Descargar la imagen iso y copiarla dentro de la memoria booteble
- Creación de un espacio libre en disco sin asignar (desde Windows). Si se desea que GNU/Linux ocupe el disco completo no es necesario este paso.
- Iniciar la computadora desde la USB booteable
- Realizar la instalación en el espacio sin asignar (o si se desea, en todo el disco).

Instalación de GNU/Linux

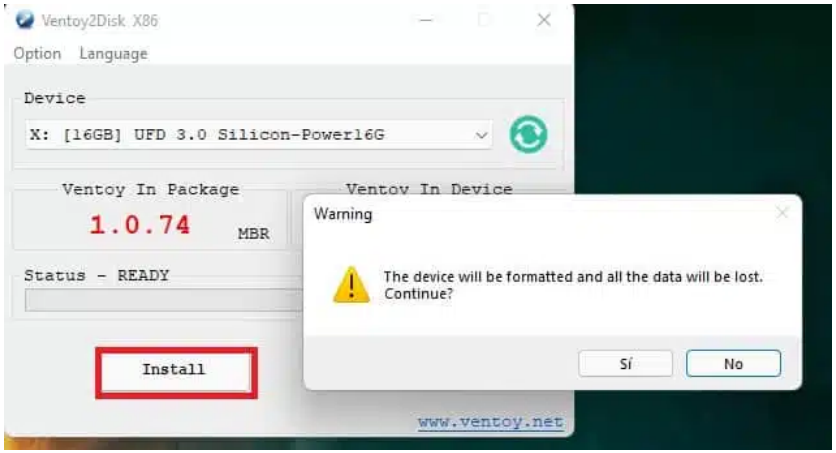
Generalmente se usan programas como Rufus, sin embargo se recomienda el uso de Ventoy. Ventoy permite copiar y pegar la ISO directamente en el pendrive, agilizando y simplificando el proceso. Además permite tener más de una imagen en una sola memoria, es de código abierto, gratuito y multiplataforma. Es compatible con archivos de imagen en formato ISO/WIM/IMG/VHD(x)/EFI y está disponible tanto para Windows como para Linux. Para hacer una USB booteable con ventoy:

- Descarga Ventoy desde el sitio oficial: link aqui
- Abre la aplicación (Ventoy2Disk.exe). En la ventana principal de Ventoy, selecciona el pendrive USB desde el desplegable "Devices".
- Después pulsa sobre "Install" para que Ventoy cree el sistema de archivos. Ten en cuenta que este proceso formateará el USB y borrará todo su contenido.
- Verás dos mensajes de advertencia en pantalla. Pulsa "Sí" para continuar.

Creación de una USB Booteble



(a) Seleccionar la unidad de almacenamiento



(b) Instalar y aceptar

Creación de una USB Booteable

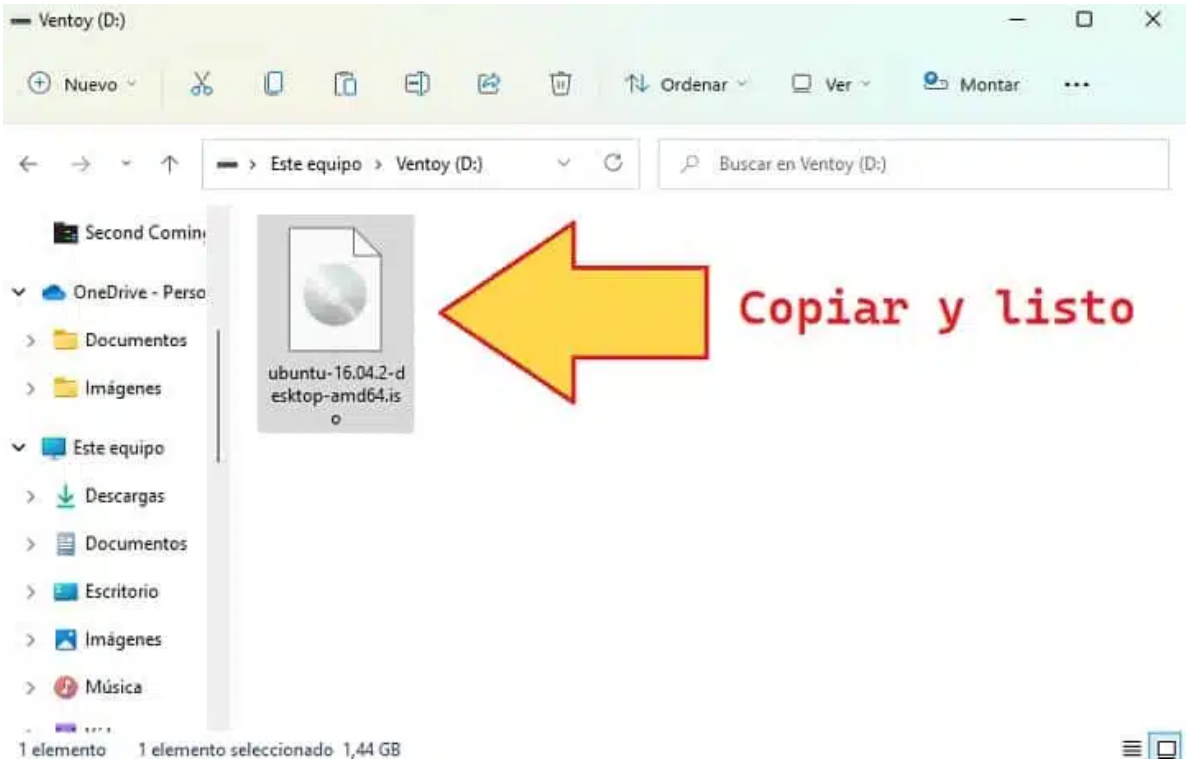
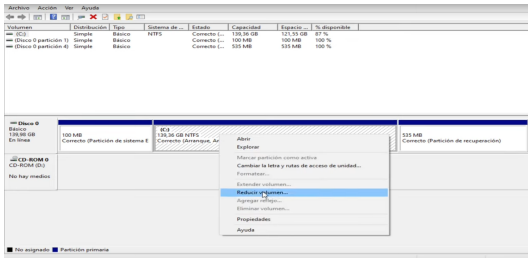


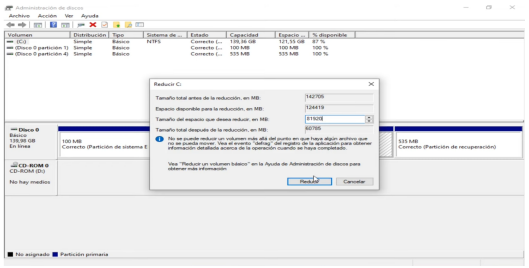
Figure: Una vez terminado el proceso, solo copia la iso dentro de la unidad de almacenamiento

Creación de un espacio libre en disco desde Windows

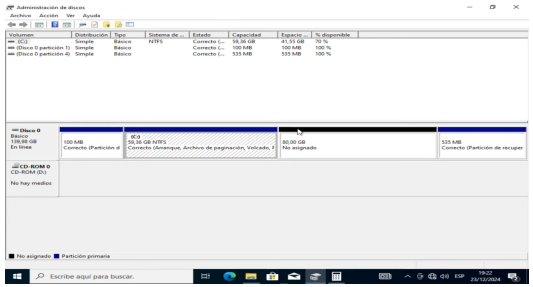
- En inicio buscamos **Administración de discos** y abrimos el programa.
- Identificamos la partición de Windows, damos click derecho y seleccionamos **Reducir volumen**.
- Colocamos el tamaño deseado en MB y damos en **reducir**.



(a) Reducir volumen



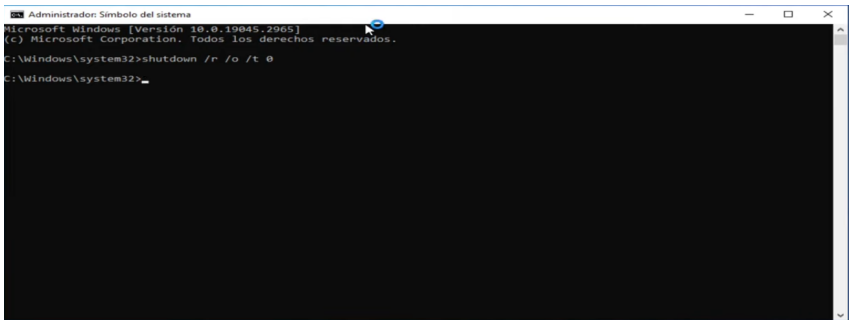
(b) Reducir



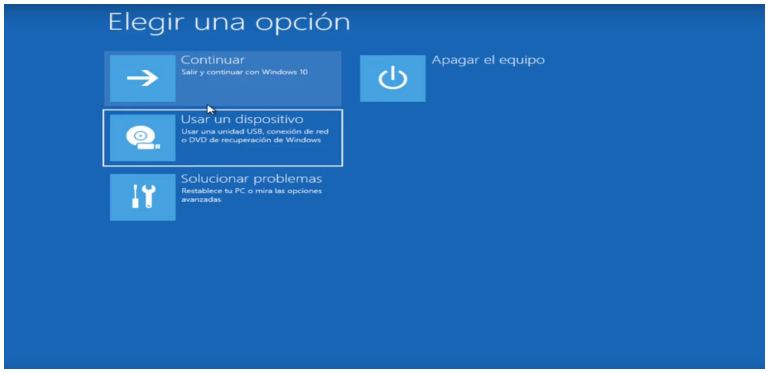
(c) Volumen

Pre-instalación

- Abrimos la terminal, símbolo del sistema, y escribimos el comando: *shutdown /r /o /t 0*
- Este comando te lleva a las opciones avanzadas de Windows, ahí puedes seleccionar "Usar un dispositivo"

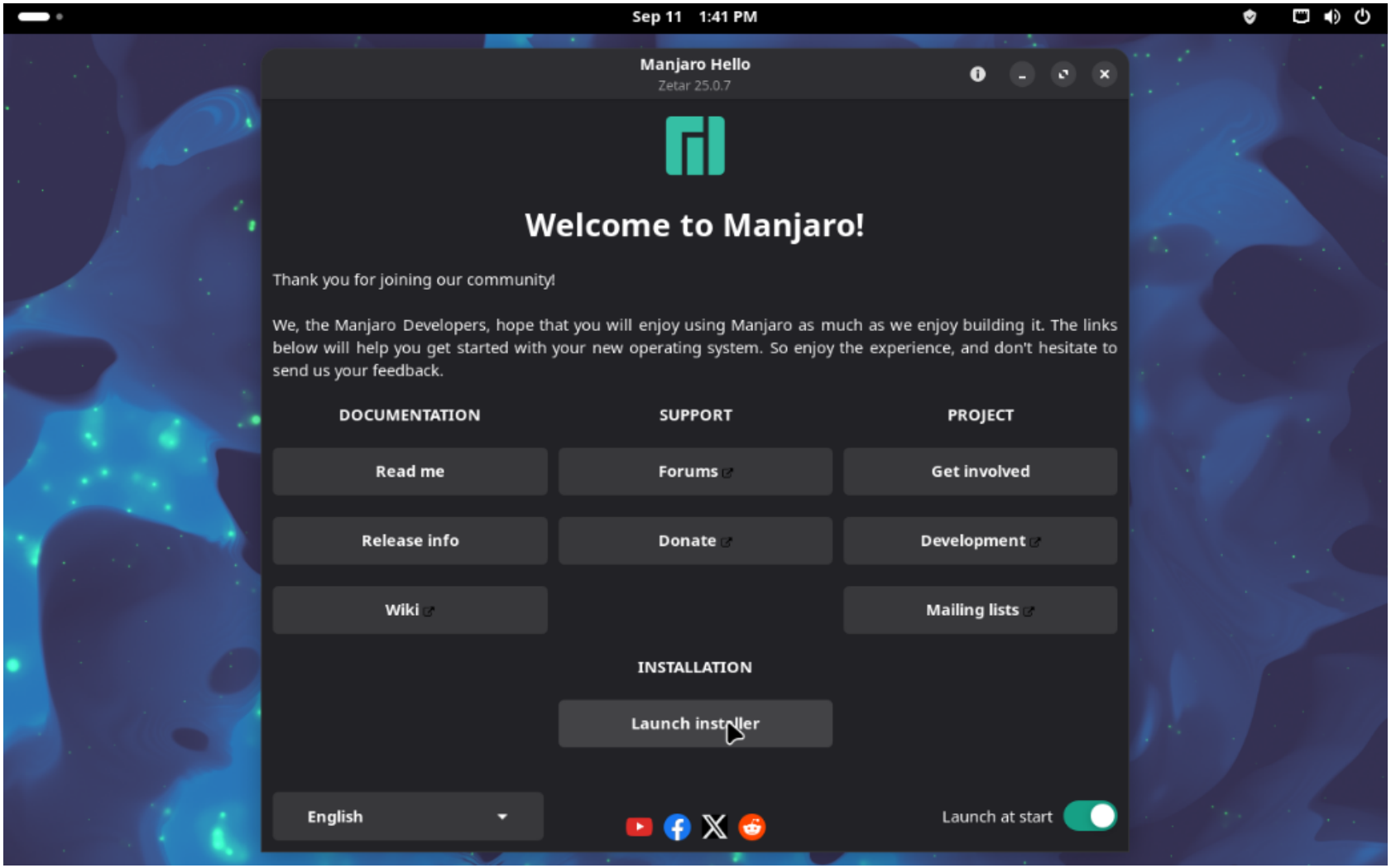


(a) Reiniciar el sistema

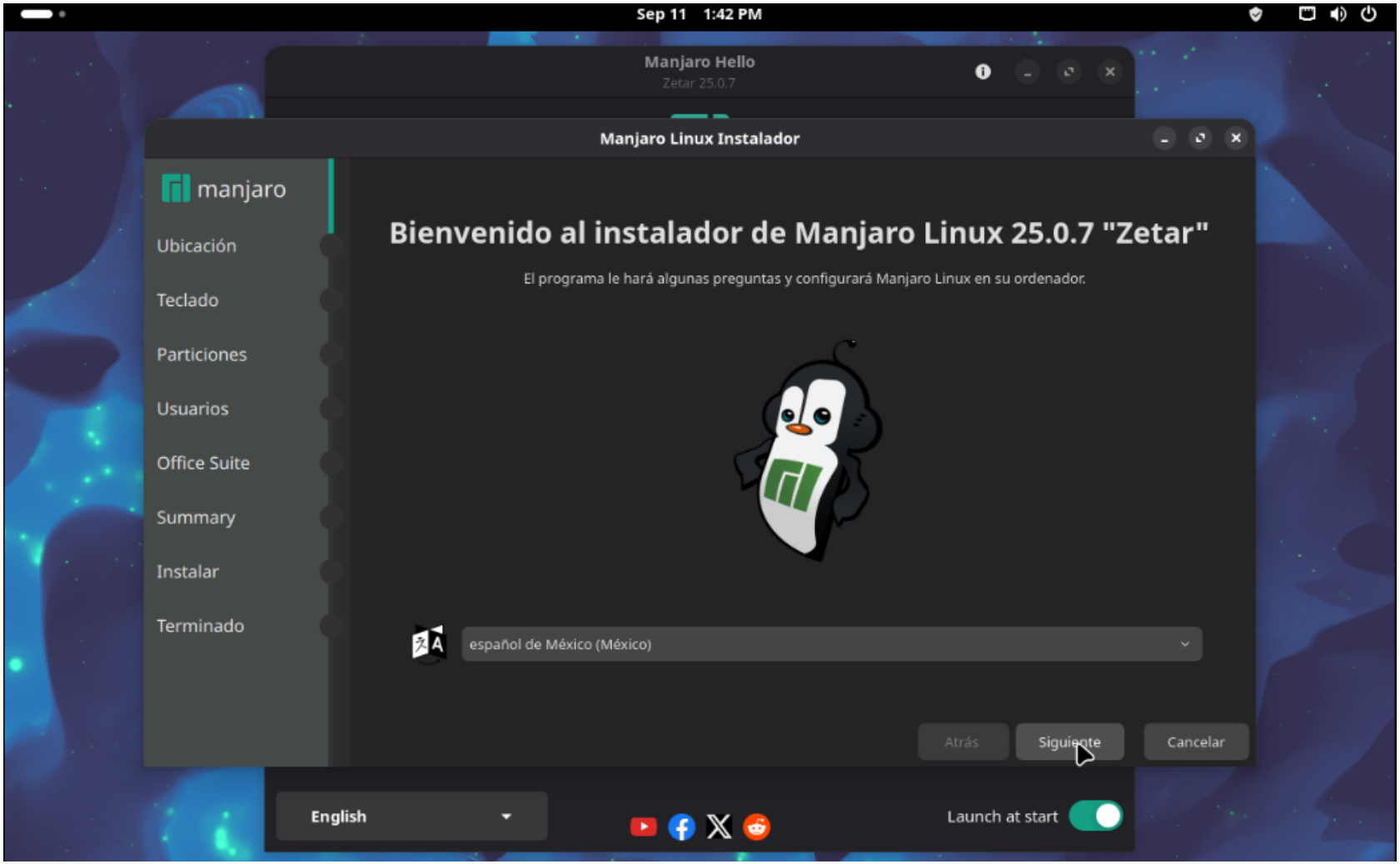


(b) Modo avanzado

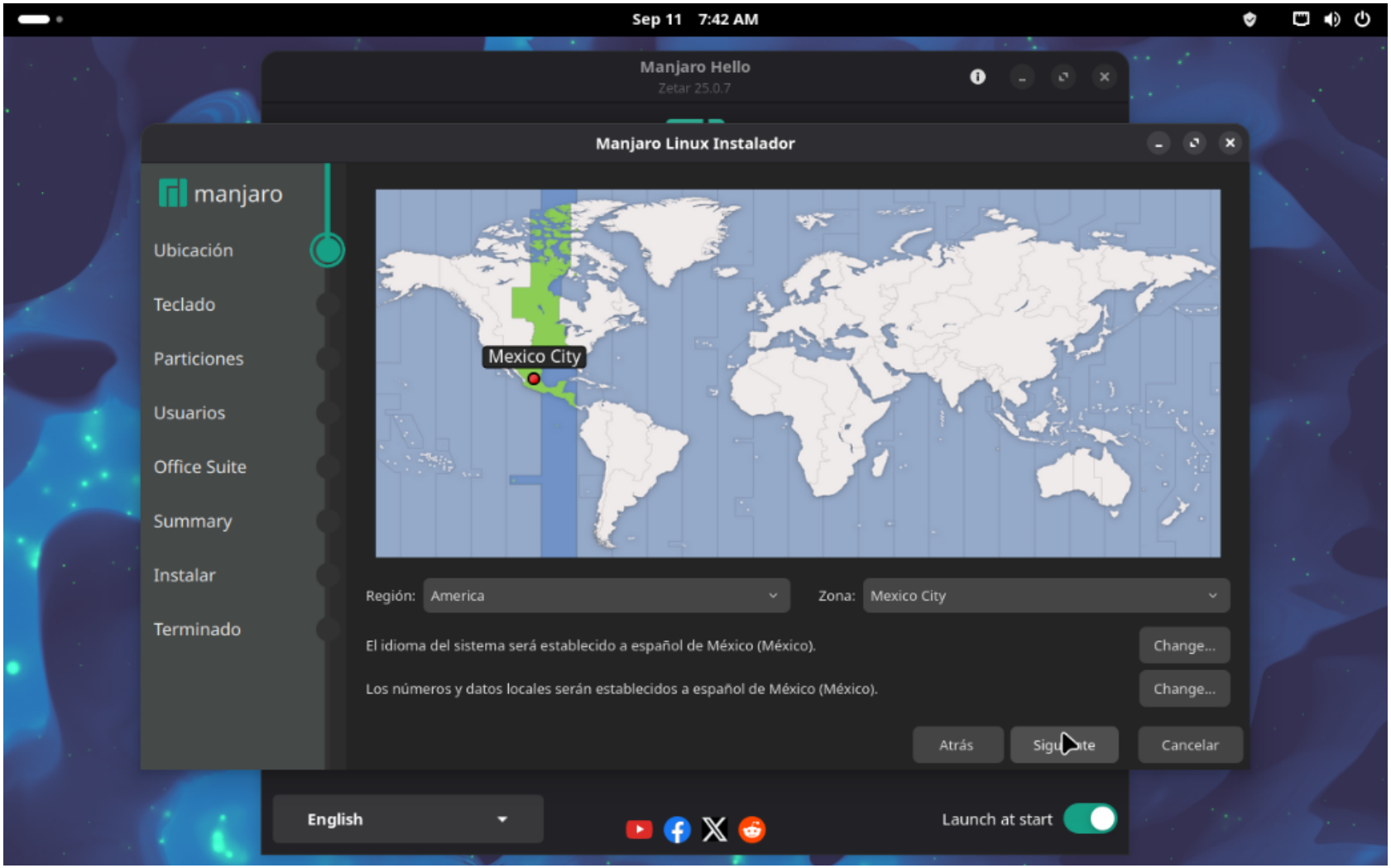
Proceso de instalación de manjaro



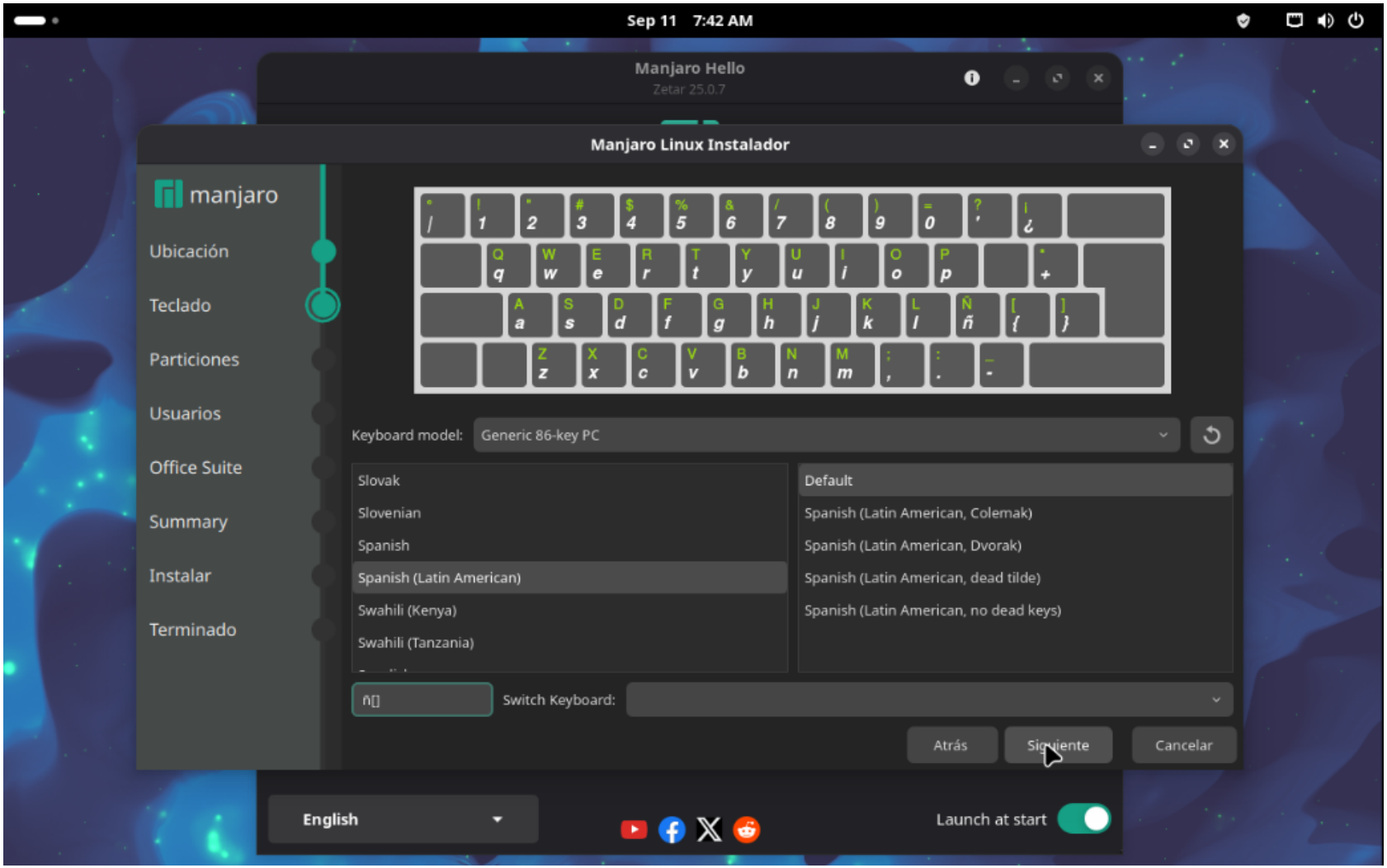
Proceso de instalación de manjaro



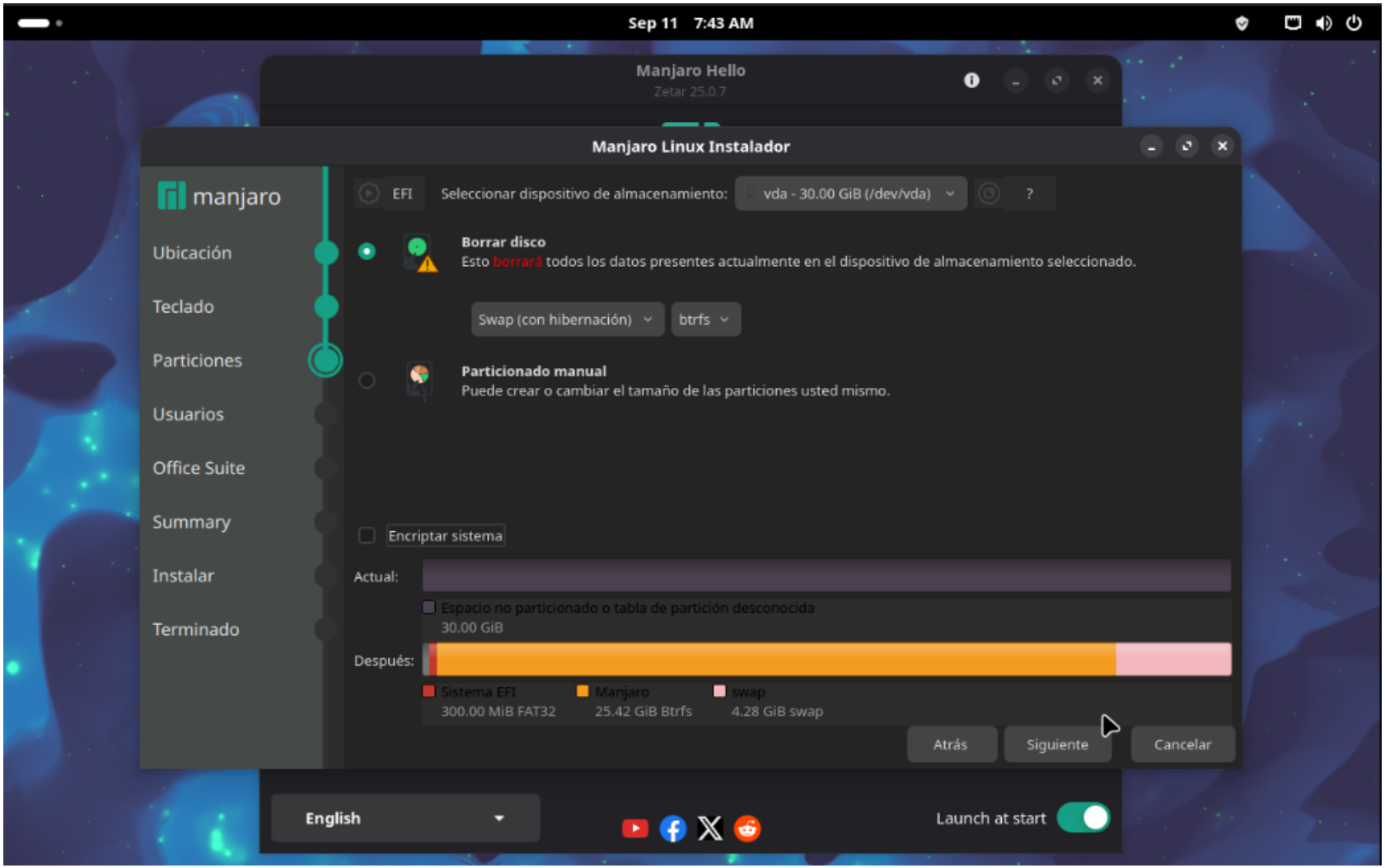
Proceso de instalación de manjaro



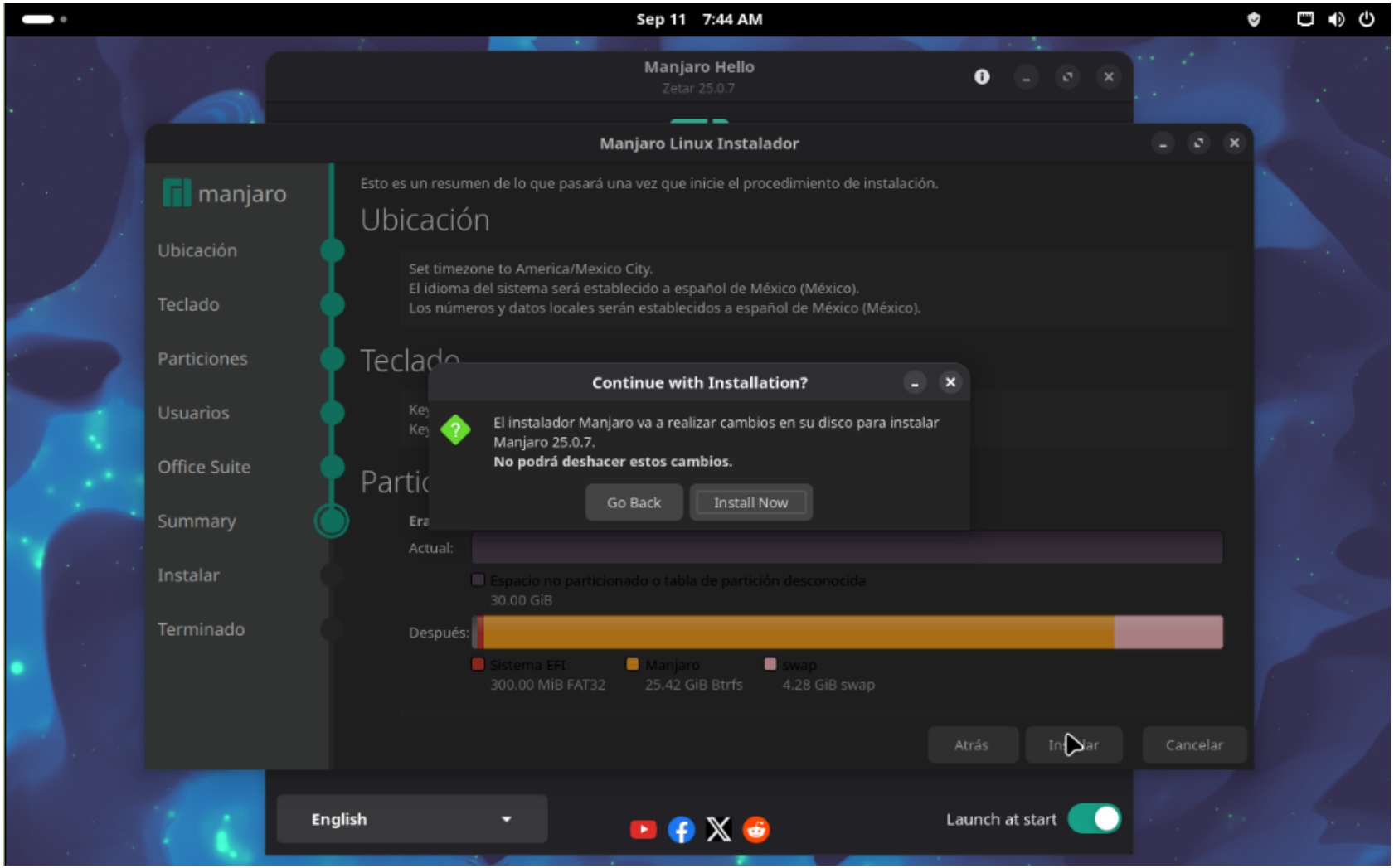
Proceso de instalación de manjaro



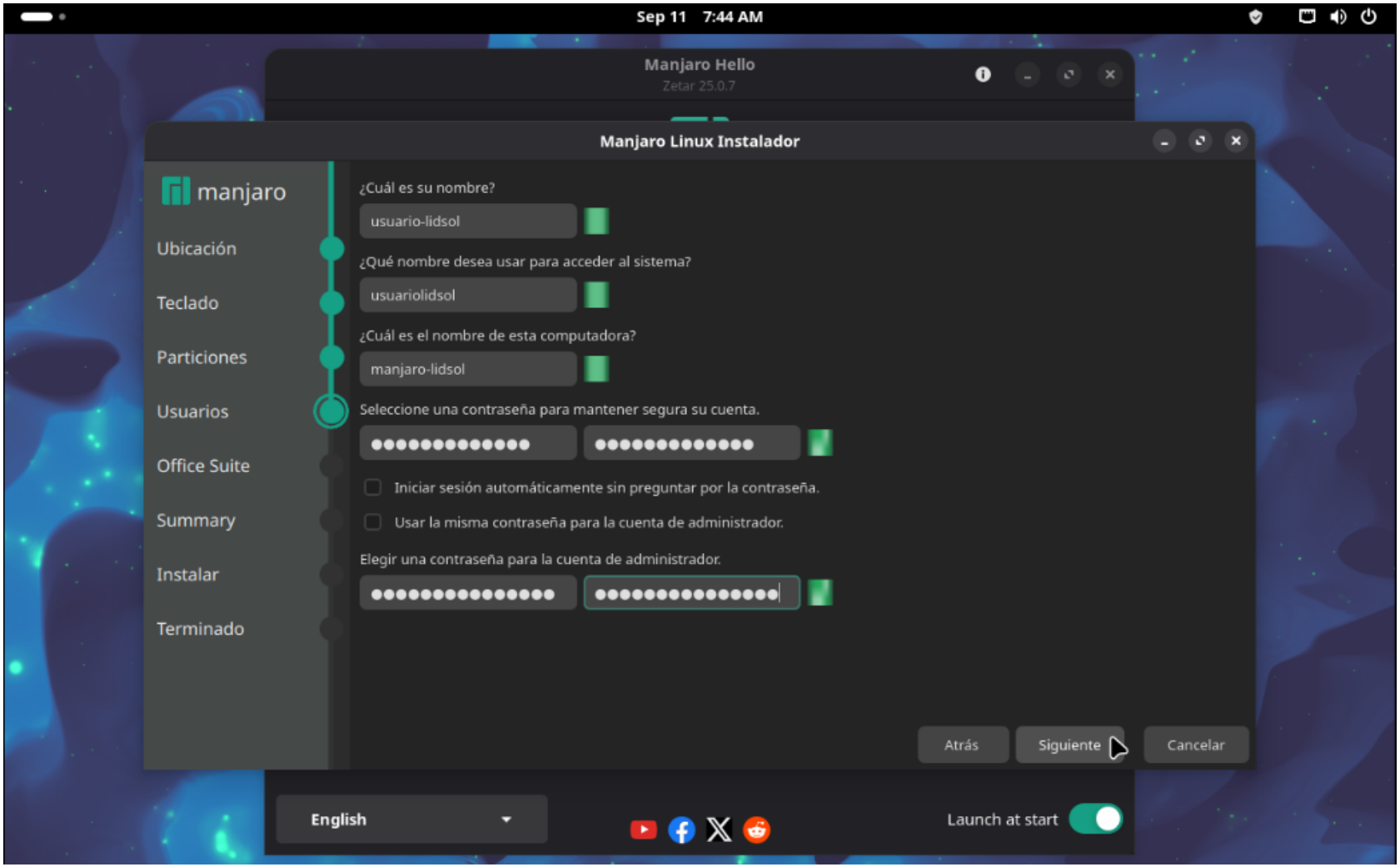
Proceso de instalación de manjaro



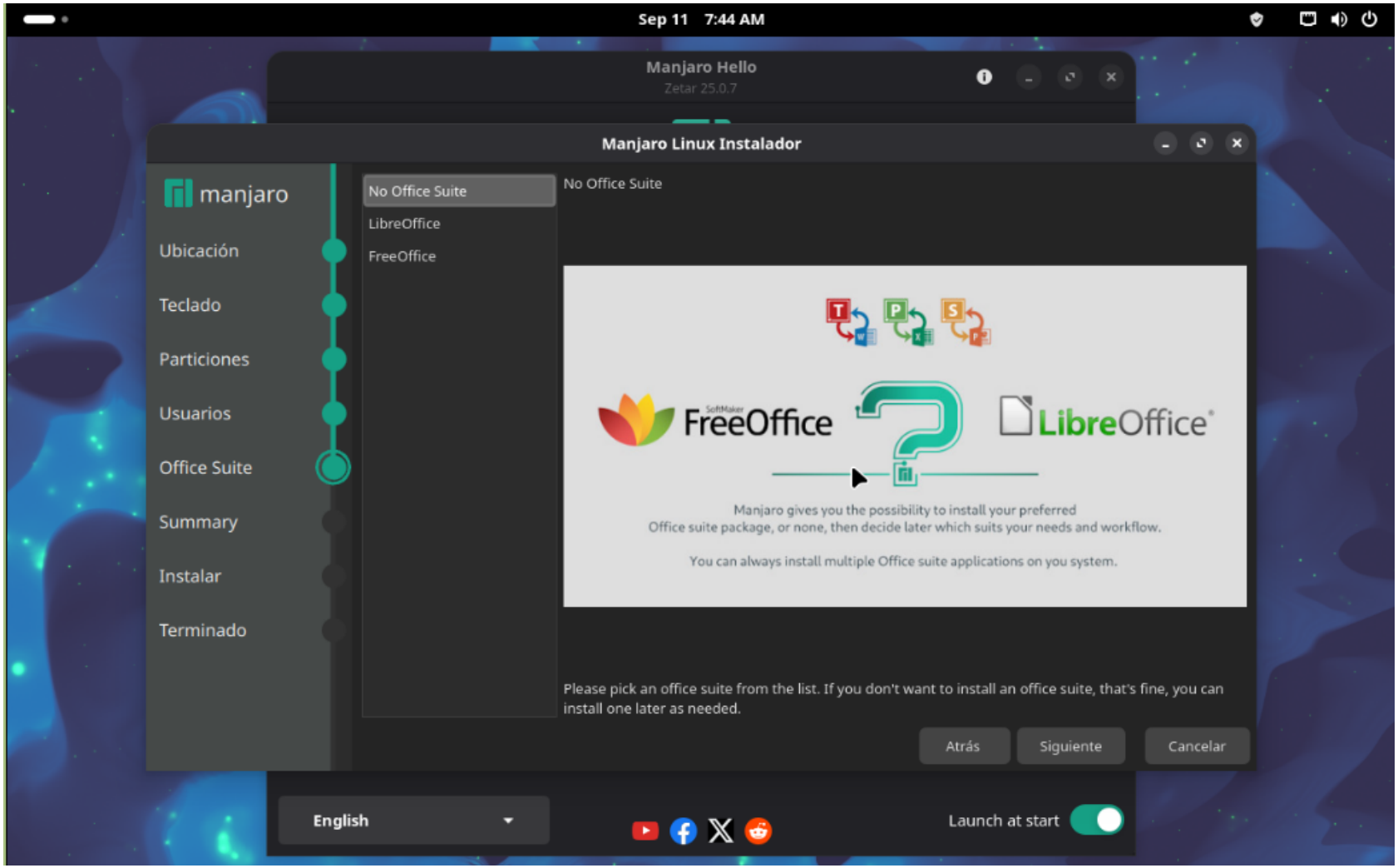
Proceso de instalación de manjaro



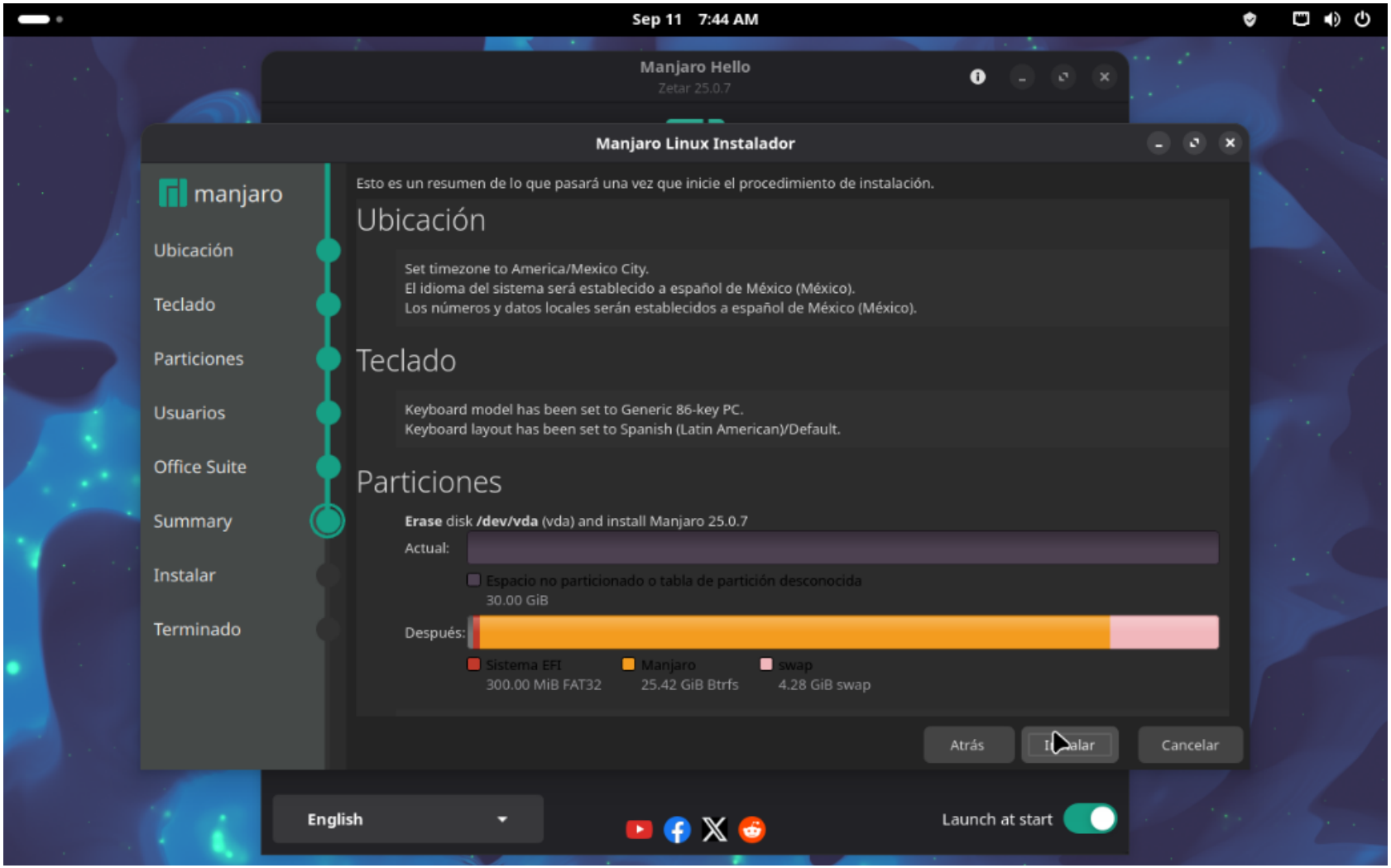
Proceso de instalación de manjaro



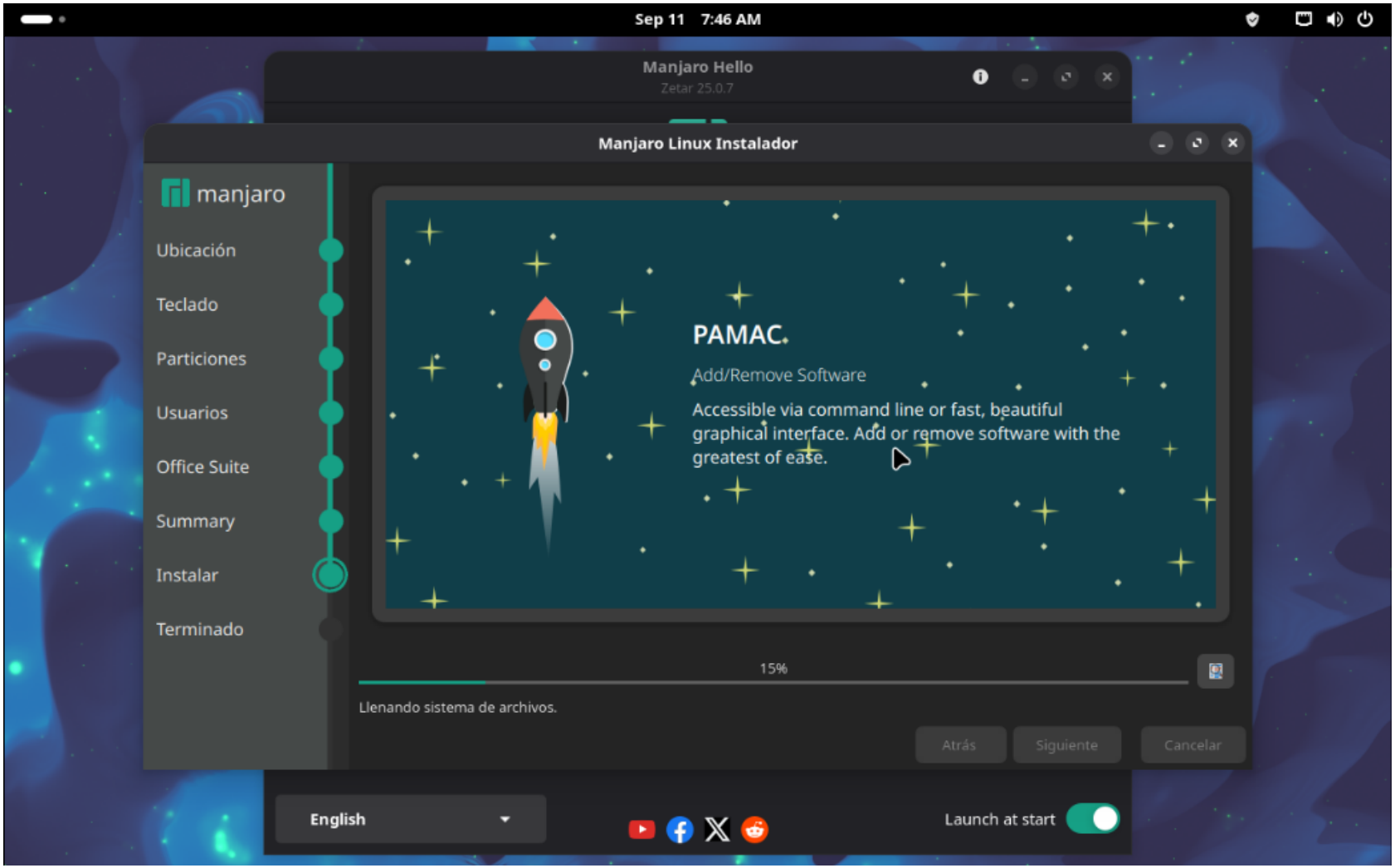
Proceso de instalación de manjaro



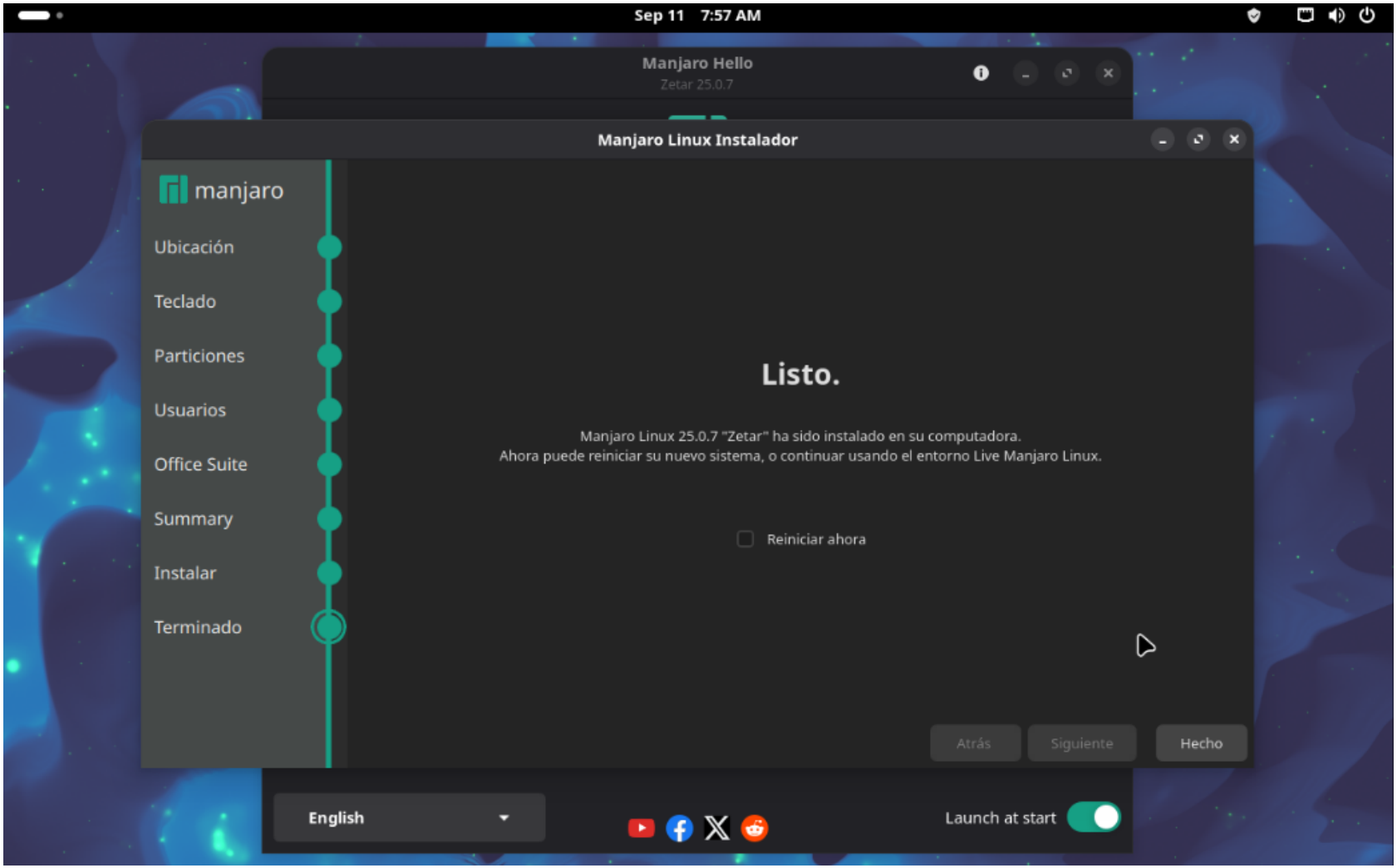
Proceso de instalación de manjaro



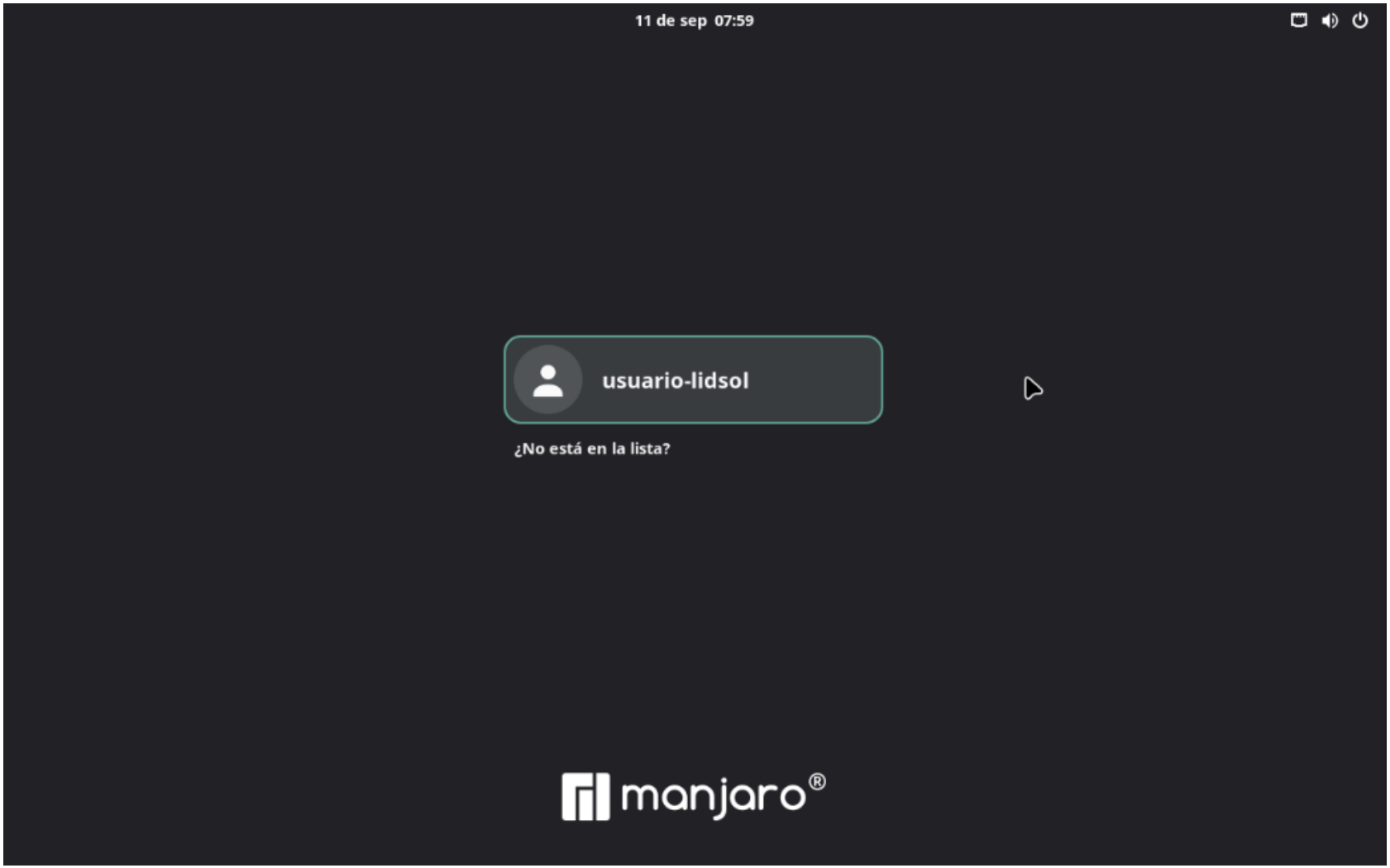
Proceso de instalación de manjaro



Proceso de instalación de manjaro



Proceso de instalación de manjaro



Instalación de Arch

Establecer la distrubución de teclado

Establecer la distribución de teclado temporal, importante para que el teclado reconozca todos los caracteres.

Para la distrubución de teclado de LATAM

```
loadkeys la-latin1
```

Para la distribución de teclado española

```
loadkeys es
```

Para la distrbución de teclado inglés Estados Unidos.

```
loadkeys us
```

Instalación de Arch

Conectarse a internet

Se puede usar un cable Ethernet. Para conectarse a una red Wi-FI, se puede se usa alguna de las siguientes órdenes:

nmcli

```
$ nmcli dev wifi -a connect 'nombre-de-red' password
    'contraseña'
```

iwd

```
$ systemctl enable --now iwd
$ iwctl --passphrase 'contraseña' station wlan0 connect
    'nombre-de-red'
```

Instalación de Arch

Verificar el modo de arranque

Verificamos si la placa base es compatible con UEFI, consultando si existe el directorio especificado y si hay archivos en él, si no se muestra información ni archivos, el arranque solo es compatible con BIOS/Legacy. Esta guía de instalación está enfocada en UEFI.

```
$ ls /sys/firmware/efi/efivars
```

Identificamos el disco en el cual queremos instalar la distribución, con la orden **lsblk**, ejemplo:.

```
$ lsblk
NAME      MAJ:MIN RM   SIZE RO TYPE  MOUNTPOINTS
sda         8:0    0 931.5G  0 disk
sdb         8:16   0 465.8G  0 disk
```

En este caso, usaremos como ejemplo "/dev/sda" para realizar la instalación, es de suma importancia identificar correctamente el disco y usar su nombre.

Instalación de Arch

Creación de particiones con cfdisk

Una vez identificado el disco, usamos **cfdisk** para crear las particiones:

```
$ cfdisk /dev/sda
```

Pasos a seguir:

- Selecciona el tipo de tabla de particiones: **GPT** (recomendado para sistemas UEFI).
- Crea las siguientes particiones:
  - 'sda1': 512 MB, tipo **EFI System** → para el arranque.
  - 'sda2': el resto del espacio (ej. 30 GB+), tipo **Linux filesystem** → raíz ('/').
  - 'sda3': 2–4 GB (o igual a la RAM), tipo **Linux swap**.
- Usa las teclas de flechas para navegar, Enter para seleccionar y "Write" para guardar.

Instalación de Arch

Dualboot: ¿ya existe una partición EFI?

Si ya tienes un sistema operativo (como Windows):

- Es probable que ya exista una partición EFI en el disco.
- Esta partición no debe eliminarse ni formatearse.
- Puedes reutilizarla para Arch, montándola en `/mnt/efi` más adelante.

Cómo verificar si ya existe una partición EFI:

- Con 'lsblk', fíjate si hay una partición de 100–512 MB, usualmente tipo 'vfat', y a veces con punto de montaje '/boot/efi':

```
$ lsblk -f
```

- También puedes verla con 'cfdisk', donde aparecerá como `EFI System`.

Si existe, solo crea las particiones necesarias para Arch:

- Raíz ('/')
- Swap

Instalación de Arch

Formateo de particiones

Usamos los siguientes comandos para formatear las particiones recién creadas, recuerda identificar correctamente el nombre de cada partición.

Formateo de la partición de arranque (solo si es nueva)

```
$ mkfs.fat -F 32 /dev/sda1
```

Atención: Si estás instalando Arch en **dualboot**, y ya existe una partición EFI (por ejemplo de Windows), **NO la formatees**. Solo móntala como ‘/mnt/efi‘ más adelante, pero **déjala intacta**.

Formateo de la partición root

```
$ mkfs.btrfs /dev/sda2
```

Formateo de la partición swap

```
$ mkswap /dev/sda3
$ swapon /dev/sda3
```

Instalación de Arch

Puntos de montaje de las particiones

Montamos las particiones en el nuevo sistema:
Montaje de la raíz (‘/‘)

```
$ mount /dev/sda2 /mnt/
```

Si es una instalación limpia: Montaje de la partición de arranque EFI
Creamos el punto de montaje y montamos la nueva partición EFI.

```
$ mkdir -p /mnt/efi
$ mount /dev/sda1 /mnt/efi
```

Si vas a hacer dualboot (por ejemplo con Windows): Identifica la partición EFI existente (usualmente ‘sda1‘, tipo ‘EFI System‘) y móntala:

```
$ mkdir -p /mnt/efi
$ mount /dev/sda1 /mnt/efi
```

No formatees esta partición si contiene el gestor de arranque de otro sistema (Windows).

Instalación de Arch

Instalación del sistema

Instalamos el sistema con la orden **pacstrap**

```
$ pacstrap /mnt linux linux-firmware linux-headers
    mkinitcpio base base-devel dhcpcd networkmanager iwd
    net-tools iw wireless_tools wpa_supplicant grub
    efibootmgr os-prober neovim nano fish
```

Si deseas paquetes para bluetooth:

```
$ pacstrap /mnt bluez bluez-utils pulseaudio-bluetooth
```

Instalación de Arch

Generar archivo fstab

Este archivo es usado para definir como las partciones se montarán de forma dinámina en el arranque:

```
$ genfstab -p /mnt >> /mnt/etc/fstab
```

Podemos verificar el contenido del archivo recién creado con:

```
$ cat /mnt/etc/fstab
```

Entramos a la raiz del nuevo sistema como root:

```
$ arch-chroot /mnt
```

Instalación de Arch

Configuración

Configuramos el dioma, para ello necesitamos editar el archivo "/etc/locale.gen" con el editor de texto de nuestra preferencia, descomentando las líneas (eliminar el símbolo #) en el o los idiomas deseados.

```
$ nano /etc/locale.gen
```

Una vez descomentadas las líneas deseadas, guardamos con Ctrl+O Generamos el idioma:

```
$ locale-gen
```

Establecer la variable LANG en locale.conf

```
$ echo LANG=es_MX.UTF-8 > /etc/locale.conf
```

Exportar la variable LANG como el local especificado

```
$ export LANG=es_MX.UTF-8
```

Instalación de Arch

Configuración

Generamos la zona horaria

```
$ ln -sf /usr/share/zoneinfo/America/Mexico_City /etc/
    localtime
```

Configuamos el reloj:

```
$ hwclock -w
```

Definimos la distribución de teclado en “vconsole.conf” para que permanezca en cada reinicio, esto solo aplica a la terminal virtual, es decir, no aplica en el entorno de escritorio o WM.

```
$ echo KEYMAP=la-latin1 > /etc/vconsole.conf
```

Instalación de Arch

Configuración

Colocamos un hostname, es decir, un nombre a la computadora.

```
$ echo nombre_de_pc > /etc/hostname
```

Ahora configuraremos a los **usuarios**. Primero establecemos la contraseña del usuario root

```
$ passwd root
```

Creamos un usuario

```
$ useradd -m -g users -G wheel -s /bin/fish
    nombre_de_usuario
```

Se esta usando fish porque lo instalamos anteriormente, de lo contrario usa /bin/bash

Ponemos una contraseña al usuario recien creado

```
$ passwd nombre_de_usuario
```

Instalación de Arch

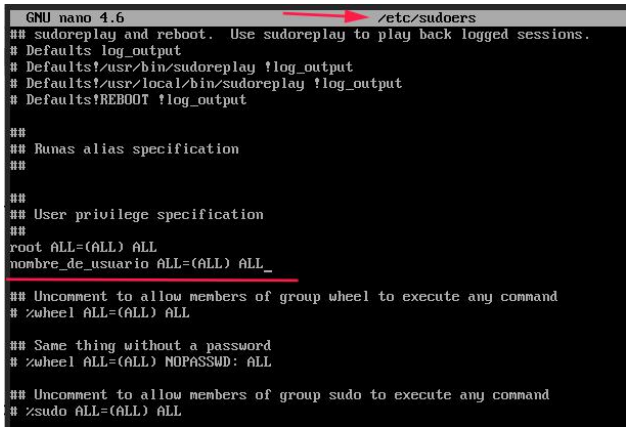
sudoers

Para darle permisos administrativos a nuestro usuario recien creado, debemos añadirlo al archivo sudoers.

```
$ EDITOR=nano visudo
```

Buscamos “root ALL=(ALL) ALL” y abajo ponemos nuestro usuario. Para que tenga permisos y mismos privilegios al ejecutar sudo.

```
$ nombre_de_usuario ALL=(ALL) ALL
```



Instalación de Arch

Instalación del grub

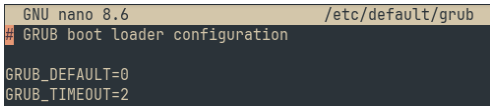
Instalamos el gestor de arranque, en la partición correspondiente:

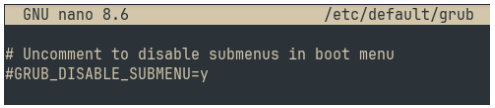
```
$ grub-install --target=x86_64-efi --efi-directory=/efi
    --bootloader-id=Arch
```

Podemos editar el archivo de configuración del grub:

```
$ nano /etc/default/grub
```

Aquí podemos cambiar el tiempo de espera del grub (en las primeras líneas con TIMEOUT) y activar os-prober para identificar otro sistema operativo instalado (hasta el final del archivo).





Dualboot: Asegúrate de activar ‘os-prober‘ en ‘/etc/default/grub‘ para que GRUB detecte automáticamente Windows u otros sistemas. Debemos modificar la última línea, simplemente descomentala o escribe: `GRUB_DISABLE_OS_PROBER=false`

Instalación de Arch

Configuración del grub - ultimas configuraciones

Generamos el archivo de configuración del grub:

```
$ grub-mkconfig -o /boot/grub/grub.cfg
```

Para mayor facilidad, instalamos el entorno de escritorio GNOME:

```
$ pacman -S gnome
```

Activamos NetworkManager, de lo contrario no tendremos conexión a internet:

```
$ systemctl enable dhcpcd  NetworkManager
```

Activamos GDM, necesario para poder iniciar sesión en GNOME (entorno gráfico)

```
$ systemctl enable gdm.service
```

Si usamos bluetooth, activamos:

```
$ systemctl enable bluetooth
```

Instalación de Arch

Salimos de LiveCD

```
$ Exit
$ Exit
```

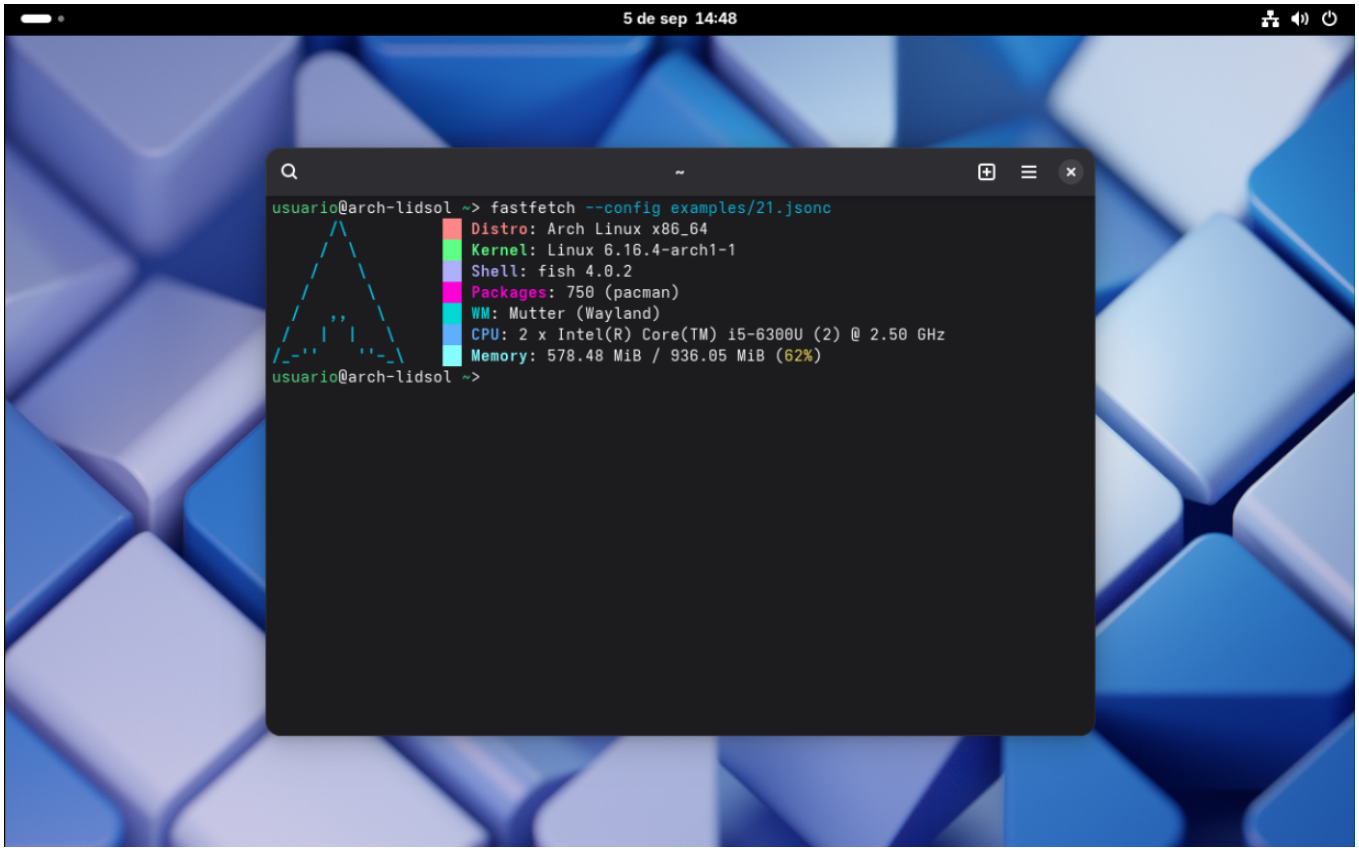
Salimos del sistema, desmontando todas las particiones:

```
$ umount -R /mnt
```

Reiniciamos el sistema:

```
$ reboot
```

Si todo salió correctamente, podremos iniciar sesión en Arch con GNOME en el próximo reinicio:



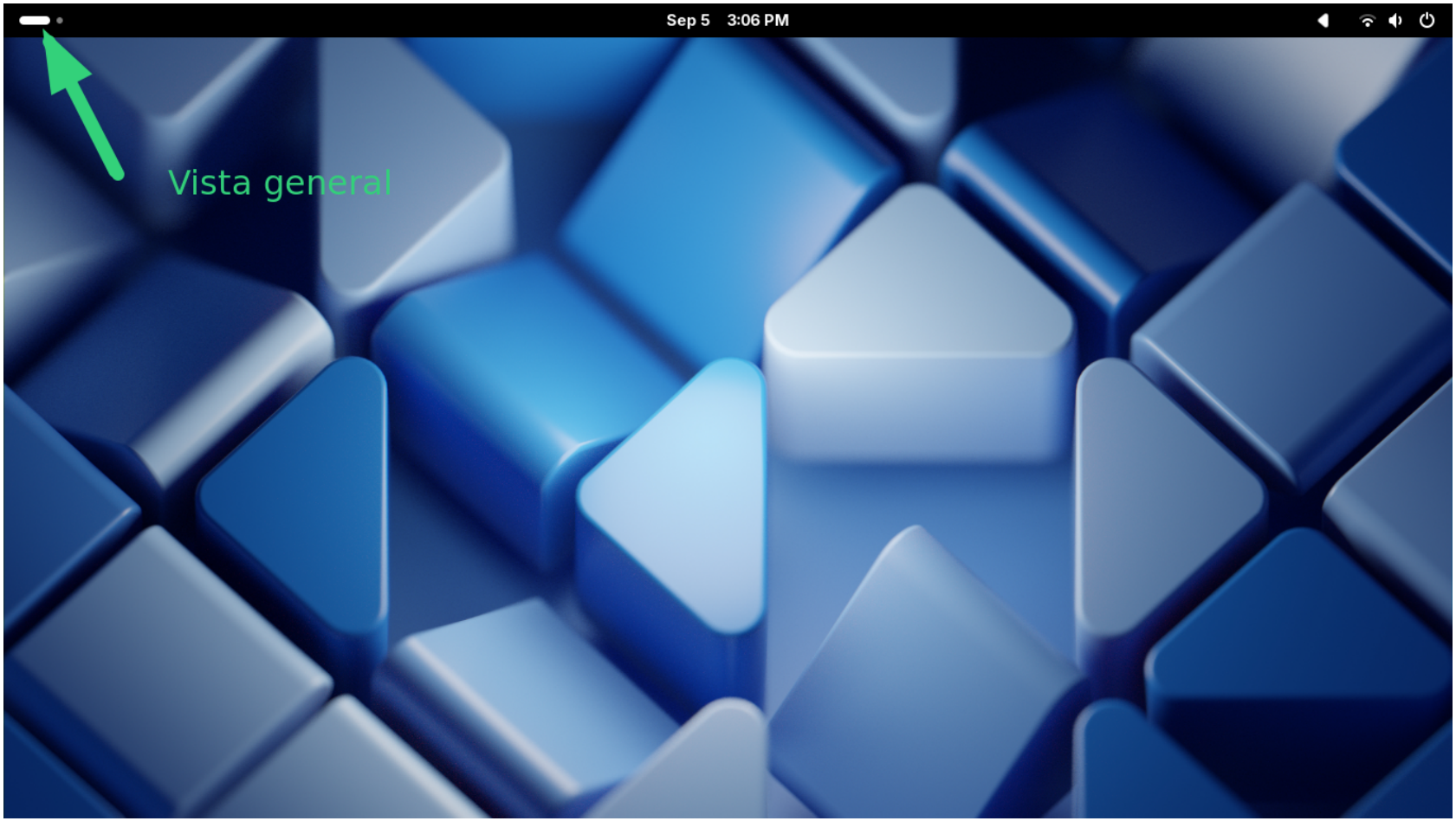
Gnome

Entorno de escritorio GNOME

GNOME es un entorno de escritorio, tiene la finalidad de ser simple y fácil de usar, está diseñado por el Proyecto GNOME y está compuesto completamente por software libre y código abierto. Gnome es parte del Proyecto GNU. La pantalla predeterminada es Wayland en lugar de Xorg. El escritorio base consiste en GNOME Shell, un complemento para el gestor de ventas Mutter. Para movernos por este entorno de escritorio, debemos entender los siguientes conceptos:

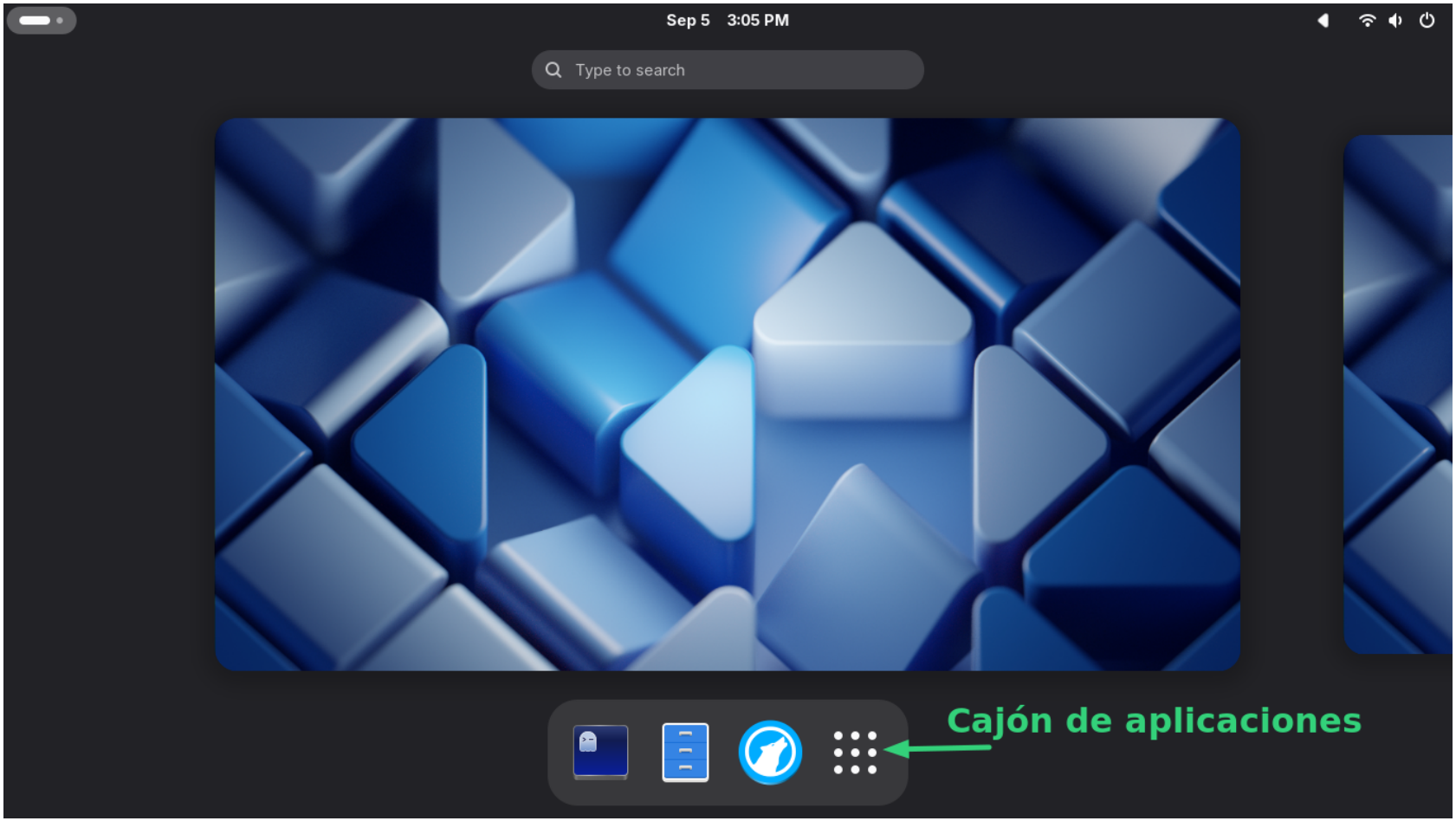
Gnome

Pantalla de inicio de GNOME



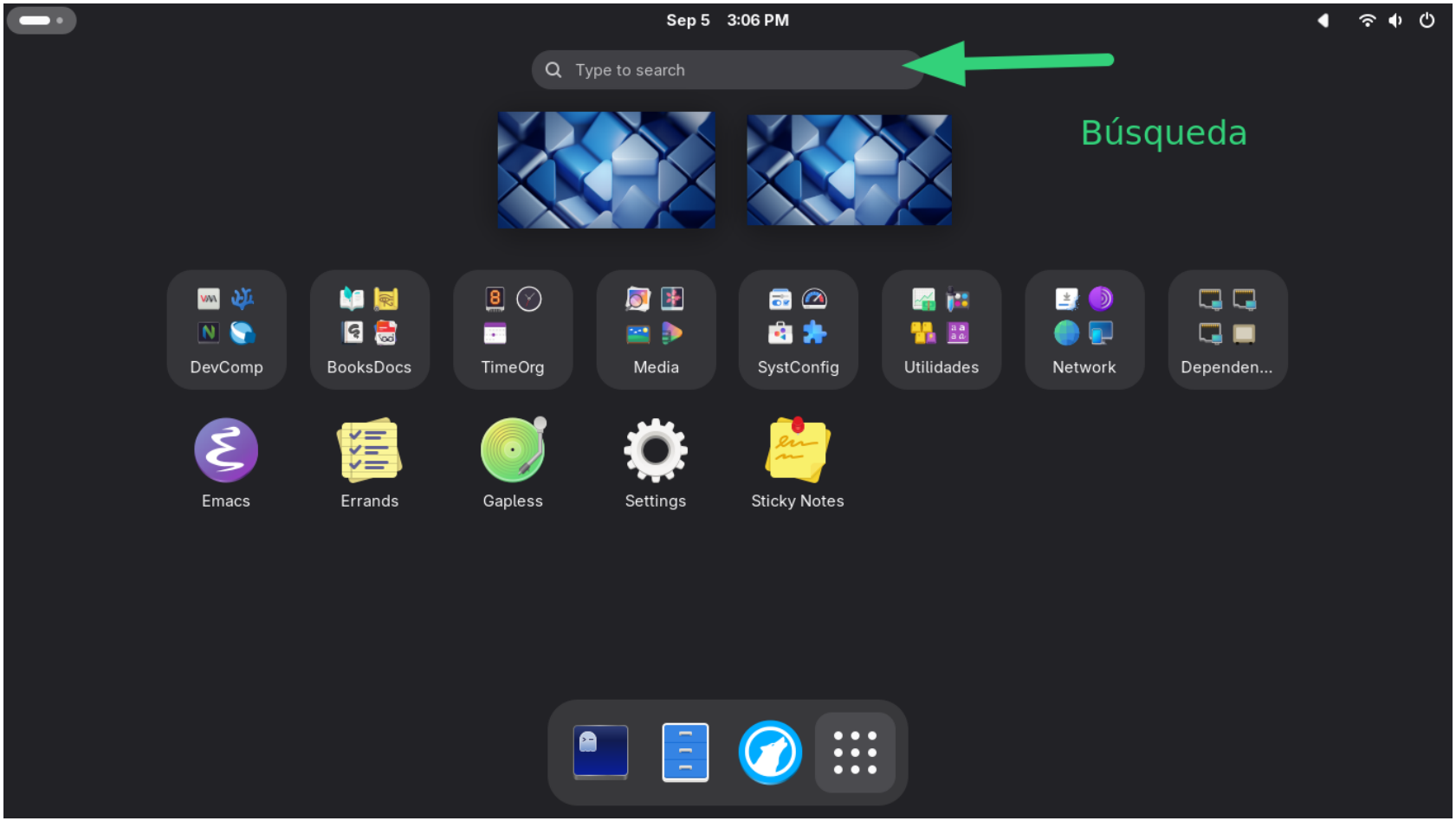
Gnome

Vista general de GNOME



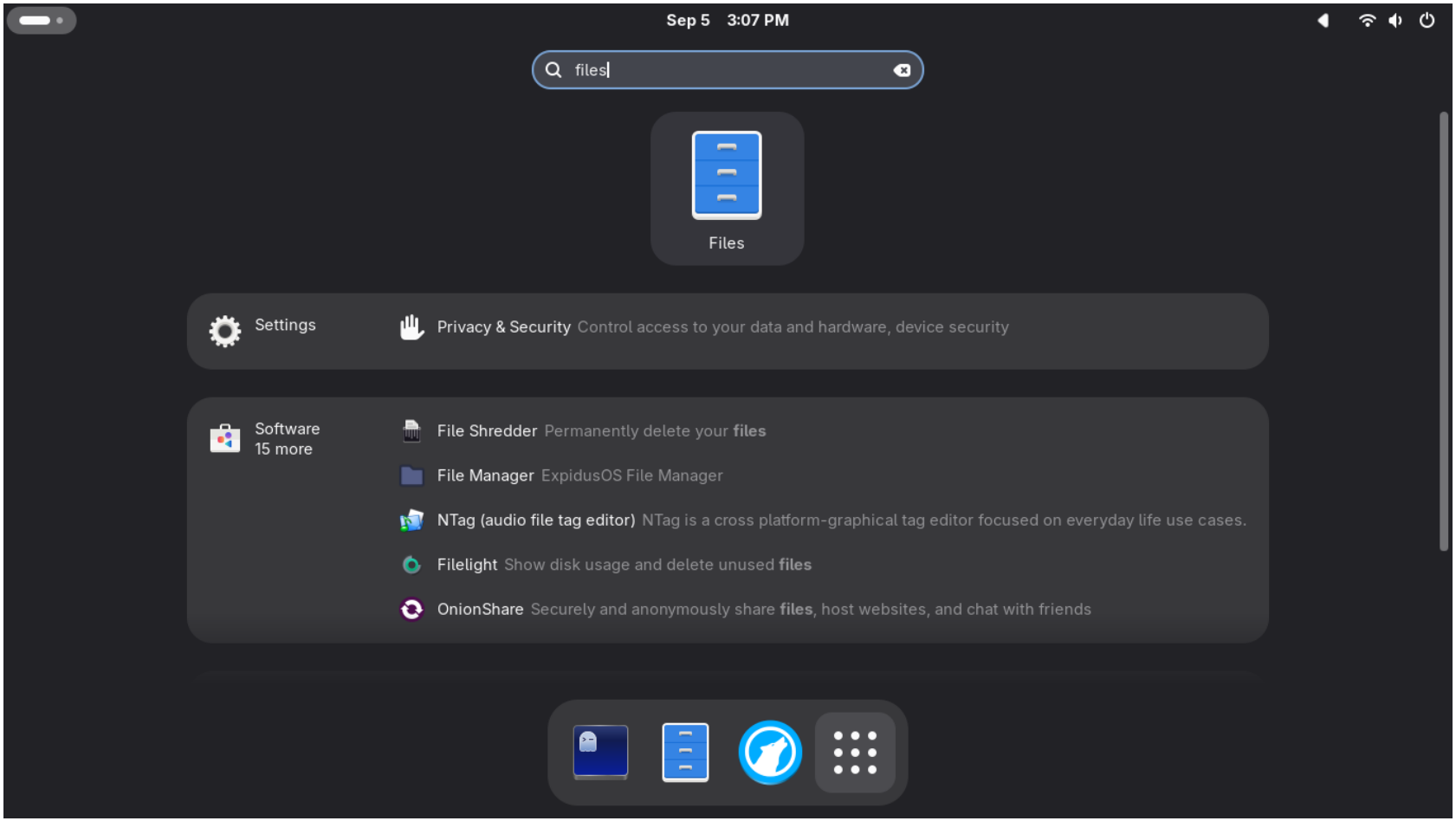
Gnome

Cajón de aplicaciones



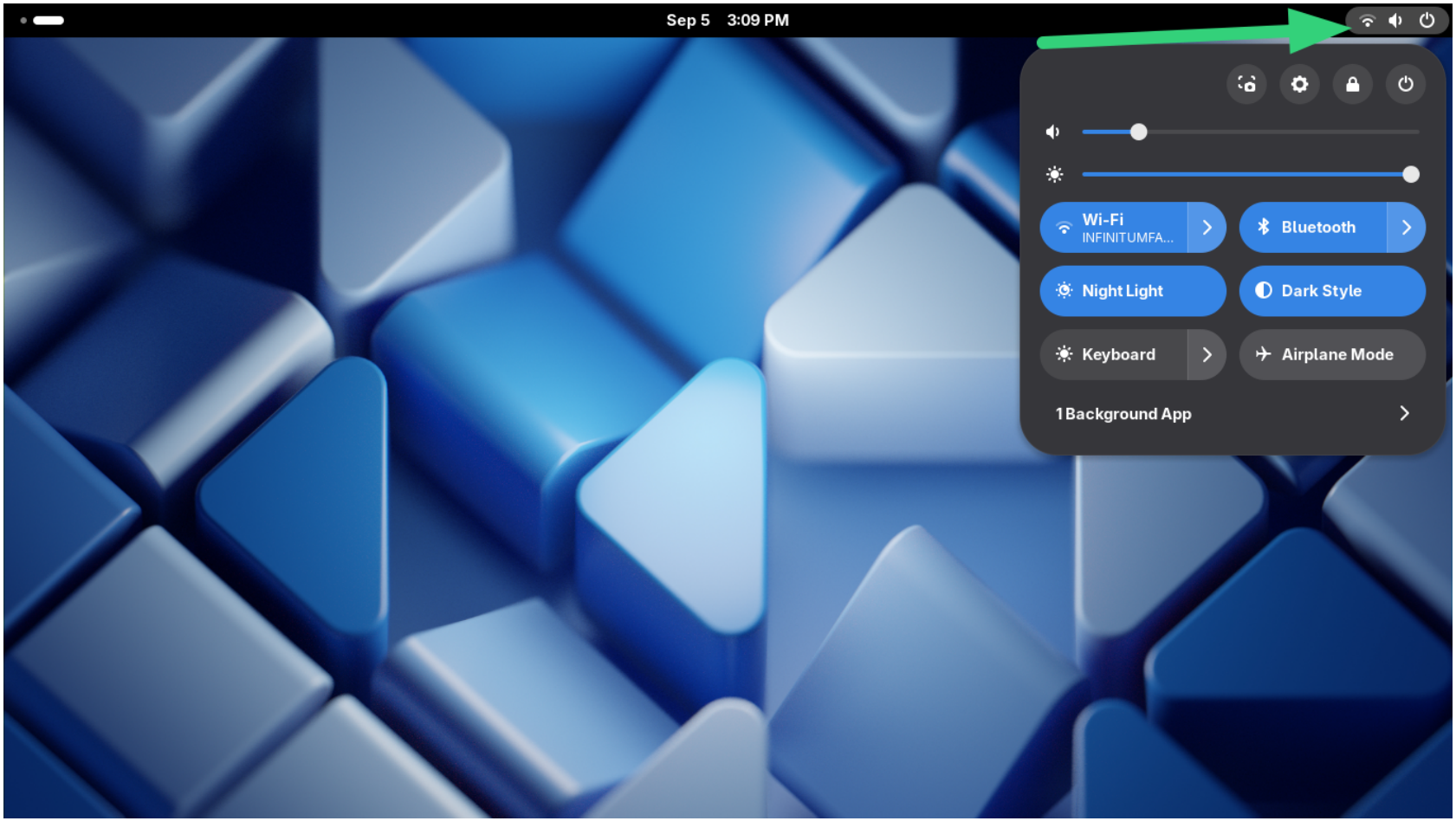
Gnome

Busqueda de apps, archivos, configuraciones, web



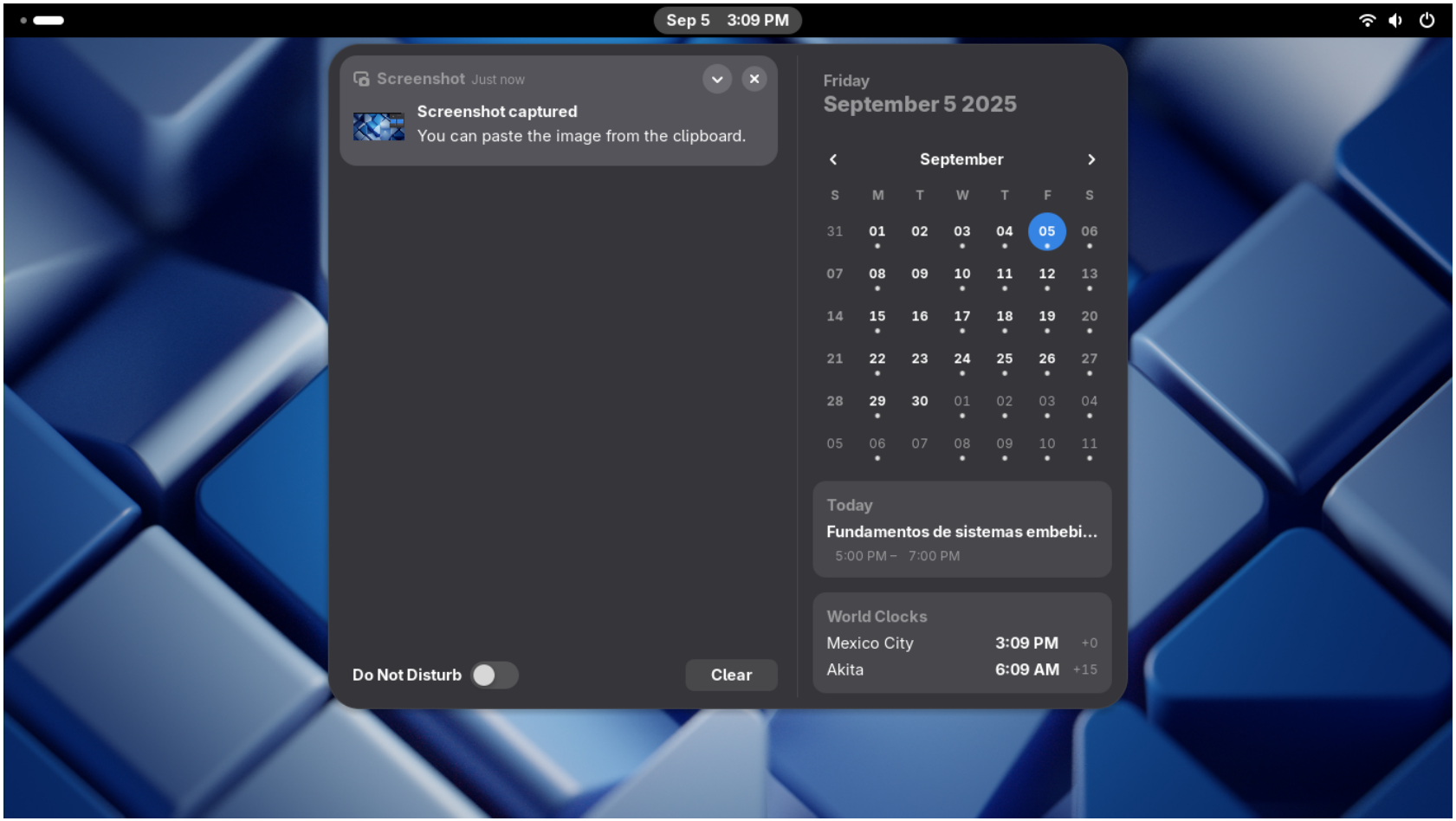
Gnome

Quicksettings - Configuraciones rápidas



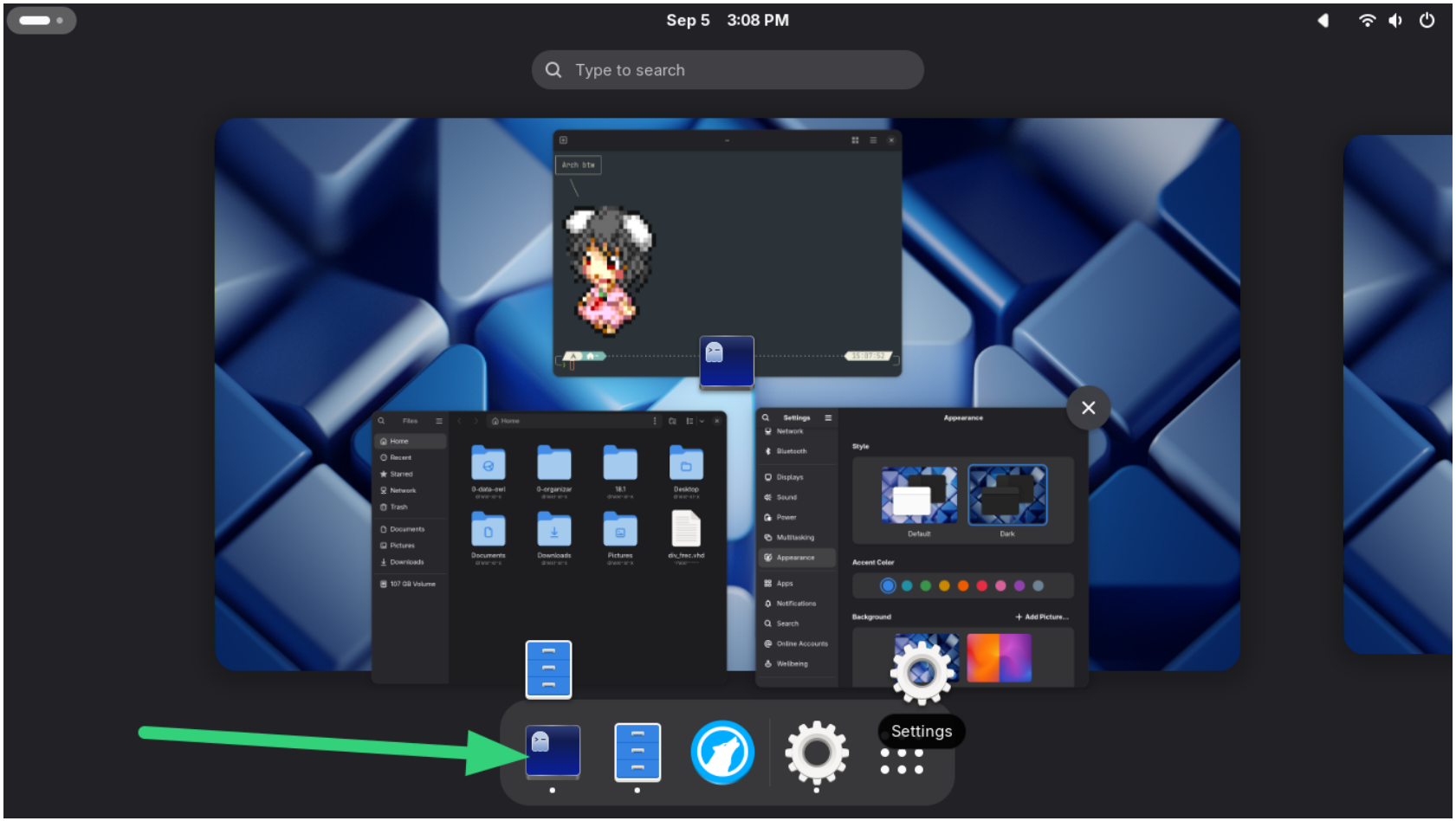
Gnome

Notificaciones y calendario



Gnome

Dock, muestra las aplicaciones gráficas en ejecución, el usuario puede anclar sus aplicaciones principales



Gnome

Workspace - Es posible usar varios espacios de trabajo para trabajar un multiples programas



Gnome

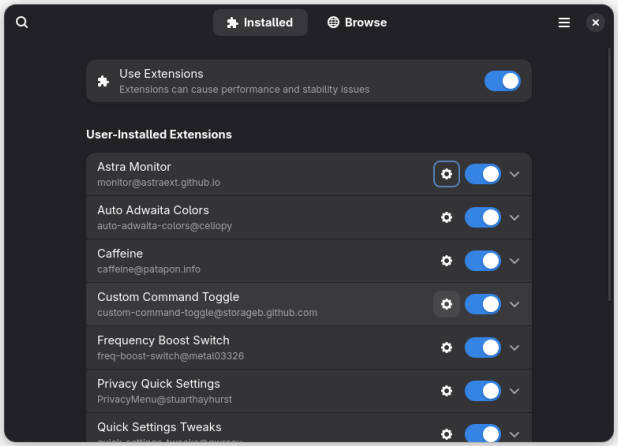
Extensiones

El entorno de escritorio GNOME se puede configurar (más allá de las configuraciones que el escritorio tiene por defecto) con la ayuda de extensiones. Las extensiones de GNOME son pequeños complementos o "addons" que se pueden instalar para modificar o ampliar su funcionalidad y apariencia. Esto se puede hacer desde el navegador web o con la ayuda de un programa que instalaremos a continuación, desde la terminal escribimos:

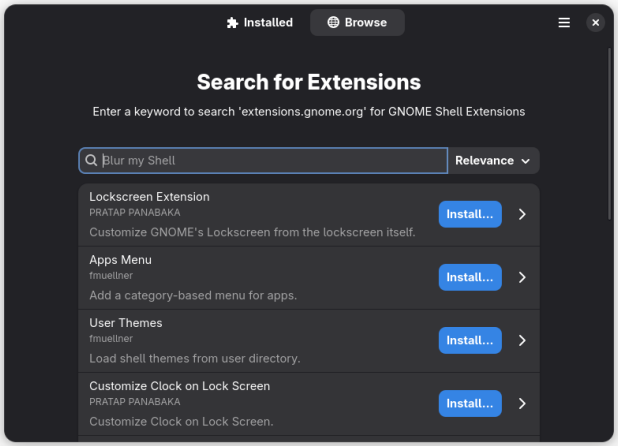
```
1 $ sudo pacman -S flatpak
2 $ flatpak remote-add --if-not-exists flathub https://
    flathub.org/repo/flathub.flatpakrepo
3 $ flatpak install flathub com.mattjakeman.
    ExtensionManager
```

Extension Manager

Después podremos encontrar en nuestro cajón de aplicaciones un nuevo programa llamado "Extension Manager", desde ahí podemos instalar extensiones.



(a) Extensiones instaladas y activadas



(b) busqueda de extensiones para instalar

Figure: Interfaz de Extension Manager

Extensiones para GNOME

Aquí algunas recomendaciones de extensiones:

- Dash To Dock: Opciones para el dock, tanto configuraciones visuales como de comportamiento.
- Dash To Panel: Oculta la barra de estado superior negra y el dock, y muestra un panel similar al de Windows, altamente personalizable.
- Blur My Shell: En el cajon de aplicaciones se muestra el fondo de pantalla difuminado y la barra de estado se vuelve opaca.
- ArcMenu: Añade un menú similar al "Start" de Windos, además de una gran cantidad de menus de incio más.
- Just Perfection: Una gran cantidad de opciones de configuración, desde el tamaño de los íconos, la barra de estado o la posibilidad de ocultar elementos del escritorio.
- Astra Monitor: Añade un pequeño widget de monitor del sistema y de los recursos de hardware en la barra de estado, hay muchass extensiones de este tipo, pero esta es el más personalizable.
- OpenBar: Añade una gran cantidad de configuraciones para la barra de estado.

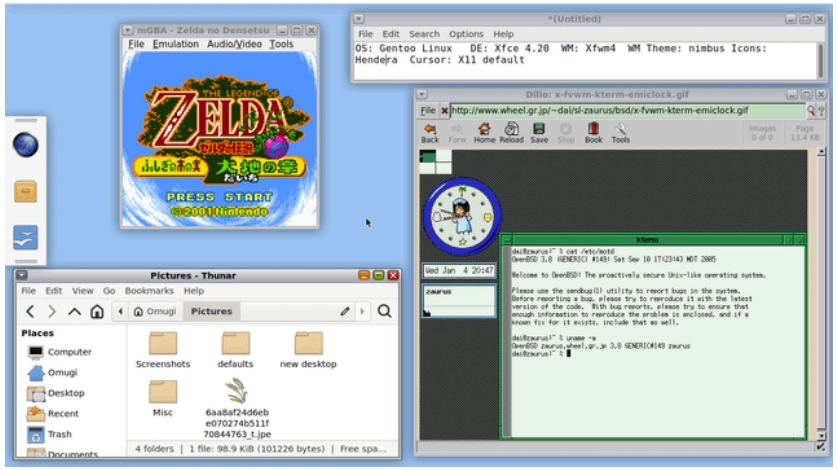
Entornos de escritorio

Es importante destacar que GNOME no es el único entorno de escritorio disponible; existen numerosas alternativas, cada una con su propio estilo visual y filosofía. Esta variedad es un reflejo de la filosofía del software libre, que se basa en el derecho a elegir y a crear opciones, promoviendo así la diversidad. Algunas de las alternativas a GNOME incluyen XFCE4, que es ligero, clásico y sencillo, y KDE Plasma, que ofrece una amplia gama de opciones y configuraciones, basado en Qt. También están los gestores de ventanas en mosaico, que son minimalistas, ligeros y altamente personalizables, como Hyprland, Sway o DWM.

Entornos de escritorio



(a) WM Hyprland



(b) DE XFCE4

Historia de GNU GNU/Linux

Historia de Unix

Los orígenes del software libre moderno

Unix fue desarrollado en los laboratorios Bell de AT&T en los años 60s y 70s. Marcó el inicio de los sistemas operativos portables, multitarea y multiusuario.

- 1969: Ken Thompson y Dennis Ritchie crean Unix en lenguaje ensamblador.
- 1973: Unix se reescribe en C, facilitando su portabilidad.
- Se crean múltiples variantes: BSD, System V, etc.
- Sirve de base para la filosofía de diseño modular y reutilizable.

Historia de GNU

El nacimiento del software libre

GNU es un acrónimo recursivo de “GNU’s Not Unix”. Fue iniciado por Richard Stallman en 1983 como una alternativa libre a Unix.

- 1983: Stallman anuncia el Proyecto GNU.
- 1985: Se funda la Free Software Foundation (FSF).
- Desarrollo de herramientas: GCC (compilador), Bash, coreutils, etc.
- Faltaba el componente central: el kernel (Hurd estaba en desarrollo).

Historia de Linux

El núcleo que lo cambió todo

Linux es un núcleo (kernel) creado por Linus Torvalds en 1991, inspirado en Minix, un sistema Unix-like educativo.

- 1991: Linus Torvalds publica la primera versión del kernel Linux.
- Fue lanzado bajo la licencia GPL, compatible con el software GNU.
- Rápidamente adoptado por la comunidad.
- Se combina con las herramientas GNU para formar un sistema operativo completo.

Relación entre Unix, GNU y Linux

El nacimiento de GNU/Linux

GNU/Linux es el resultado de combinar el kernel Linux con las herramientas del proyecto GNU.

- Unix inspiró el diseño del sistema GNU y del kernel Linux.
- GNU ofrecía herramientas libres pero sin kernel funcional (Hurd no estaba listo).
- Linux llenó ese vacío, y junto con GNU, nació el sistema operativo funcional.
- Popularmente se le llama “Linux”, pero su nombre técnico completo es “GNU/Linux”.

Software libre vs OpenSource

Diferencias entre Software Libre y Código Abierto

Filosofía, libertades y objetivos

Aunque ambos permiten acceso al código fuente, sus fundamentos son distintos:

Software Libre (Free Software)

Definido por la Free Software Foundation (FSF), se basa en la defensa de la libertad del usuario. Un programa es libre si garantiza estas **4 libertades esenciales**:

1. **Libertad 0:** Usar el programa con cualquier propósito.
2. **Libertad 1:** Estudiar cómo funciona y adaptarlo a tus necesidades (requiere acceso al código fuente).
3. **Libertad 2:** Distribuir copias para ayudar a otros.
4. **Libertad 3:** Mejorar el programa y liberar esas mejoras para beneficio de la comunidad.

El enfoque es ético y social: *la libertad es más importante que la comodidad o el rendimiento*.

Código Abierto (Open Source)

Promovido por la Open Source Initiative (OSI), prioriza los beneficios técnicos y de desarrollo colaborativo:

- Favorece modelos de desarrollo abiertos para mejorar calidad, seguridad y eficiencia.
- Permite usos comerciales y licencias más permisivas.
- Menos énfasis en la libertad del usuario como derecho.

En lugar de hablar de "libertades", destaca la **transparencia y colaboración**.

Las licencias determinan cómo se pueden usar, modificar y redistribuir los programas. Aunque muchas licencias son aceptadas tanto por la FSF como por la OSI, difieren en sus enfoques y restricciones.

Licencia GPL (General Public License)

- Creada por la Free Software Foundation. - Es la licencia libre más usada y representativa del movimiento del software libre. - Garantiza las 4 libertades del software libre. - Usa un mecanismo llamado **copyleft**:

- Las modificaciones y derivados también deben estar bajo GPL.
- Evita que alguien haga software privativo a partir de software libre.

- Ejemplo de uso: GNU, Linux, VLC, GIMP.

Licencias permisivas (Open Source)

Licencias aceptadas por la OSI que permiten mayor flexibilidad, incluso para integrar el código en software privativo:

- **MIT:** Muy simple y permisiva, permite reutilización casi sin restricciones.
- **Apache 2.0:** Similar a MIT, pero con protección de patentes.
- **BSD:** Permisiva, usada ampliamente en sistemas operativos como FreeBSD.

No usan copyleft. Su prioridad es facilitar la adopción, incluso en entornos comerciales.

Sistema de archivos

El sistema de archivos

Conceptos

Defínción genérica de **archivo**: Conjunto de datos con un nombre asociado. Suelen residir en dispositivos de almacenamiento secundario. La razón para asignar un nombre a cada archivo es que de este modo tanto como los usuarios como los programas en ejecución pueden hacer referencia a los mismos de forma lógica.

El sistema operativo dispone de un conjunto de funciones llamadas "system call"; engloba funciones relacionadas a la manipulación de procesos, dispositivos y **archivos**. Los procesos (programas en ejecución) pueden pueden escribir datos en los archivos (write) o leerlos (read). Los procesos tienen la posibilidad de crear archivos, añadir o eliminar la información de ellos, desplazarse en ellos, todo a partir del correspondiente conjunto de llamadas al sistema.

El sistema de archivos

Conceptos

En GNU/Linux los archivos están organizados en directorios. **Los directorios son archivos** que contienen información que permiten localizar otros archivos. Los directorios pueden contener, a su vez, nuevos directorios, los cuales se denominan como subdirectorios. A la estructura resultante de esta organización se le conoce con el nombre de estructura de árbol invertido. Todos los archivos y directorios dependen de un único directorio denominado directorio raíz o root, el cual se representa por el símbolo de *slash* (/).
Los archivos se identifican en la estructura de directorios por lo que se conoce como *pathname* o camino. Así, la cadena */etc/passwd* identifica a passwd como un elemento que cuelga de **etc** el cual a su vez cuelga del directorio raiz (/).

El sistema de archivos

Conceptos

- Ruta absoluta: Es la dirección completa de un directorio o archivo. Empienzan con "/". Ejemplo: */home/user/Desktop/*
- Ruta relativa: Es la dirección de un directorio o archivo en relación con el directorio actual en el que se encuentra el usuario. Por ejemplo, si estamos en el directorio */home/user*, la ruta relativa para acceder a la Desktop sería *Desktop/*
- Ruta de usuario: Es el directorio personal, se representa con $\sim/$. Es decir $\sim/ = /home/user$
- Referencia al directorio actual: El directorio actual se presenta con un punto "."
- Referencia al directorio padre: El directorio padre se representa como ".."

Directorios

Estructura de directorios

En Linux, el sistema de archivos está organizado en una jerarquía de directorios. Cada directorio tiene un propósito específico. Los sitemas de archivos pueden variar en su estructura, pero en su mayor parte cumplen con un estándar.

- / - El directorio raíz de toda la jerarquía del sistema de archivos, todo está anidado bajo este directorio.
- /bin - Aquí están los binarios (programas listos para ejecutarse).
- /boot - Archivos para el gestor de arranque; archivos del cargador de arranque del kernel, configuraciones del gestor de arranque.
- /dev - Archivos de dispositivos; los archivos de dispositivos representan dispositivos de hardware en el sistema. como discos duros, terminales, dipositivos de red.

- /etc - Directorio de configuración del sistema central. Configuraciones importantes como sudoers, passwd, hosts, y más.
- /home - Directorios personales para usuarios, contiene tus documentos, archivos, configuraciones, etcétera.
- /lib - Contiene archivos de biblioteca que los binarios pueden usar. Estas bibliotecas son esenciales para el funcionamiento del sistema.
- /media - Utilizado como punto de conexión para medios extraíbles como unidades USB.
- /mnt - Punto de montaje temporal para sistemas de archivos montados manualmente; como discos externos o particiones adicionales.
- /opt - Paquetes de software de aplicación opcionales. Contiene aplicaciones de terceros que no forman parte del sistema base. Aquí se pueden instalar programas adicionales.

- /proc - Información sobre los procesos en ejecución actualmente. Contiene archivos que representan procesos y configuraciones del kernel.
- /root - Directorio de inicio (el directorio personal) del usuario root. Es diferente a /home/root
- /run - Información sobre el sistema en ejecución desde el último arranque (datos de ejecución temporal), como información sobre procesos en ejecución y sockets.
- /srv - Datos específicos del sitio que son servidos por el sistema (como archivos de servidor web o FTP).

- /tmp - Almacenamiento para archivos temporales. Puedes crear un directorio temporar fácilmente con el comando *mktemp -d*
- /usr - Contiene software, utilidades y configuraciones del usuario. Este directorio incluye subdirectorios como ‘/usr/bin‘ (programas ejecutables), ‘/usr/lib‘ (bibliotecas), y ‘/usr/share‘ (archivos compartidos).
- /var - Directorio variable, se utiliza para el registro del sistema, seguimiento de usuarios, cachés, etc. Básicamente cualquier cosa que esté sujeta a cambios todo el tiempo.

El sistema de archivos

Conceptos

Sistema de archivos: Aquella parte del sistema operativo responsable de la administración de los datos en dispositivos de almacenamiento secundario. Debe proporcionar los medios necesarios para un almacenamiento seguro y privado de la información, y, a la vez, la posibilidad de compartir información en caso de que el usuario lo desee.

Comandos para el manejo de directorios

Comandos para el manejo de directorios

pwd

La orden **pwd** imprime el nombre del directorio de trabajo (el directorio en el cual nos encontramos).

```
$ pwd
/home/user
```

Comandos para el manejo de directorios

ls y dir

Las ordenes **ls** y **dir** muestran el contenido de un directorio. Si no se especifica un directorio, se imprime el contenido del directorio actual. Se muestra el contenido ordenado de forma alfabética si no se especifica otro orden. Su uso es el siguiente:

```
ls [OPTION]... [FILE]...
```

Ejemplos:

```
ls          # listar el directorio actual
ls carpeta/ # listar el contenido especificando una ruta
ls -l       # listar en formato largo
ls -R       # listar recursivamente
ls -r       # ordenar de forma alfabética inversa
ls -S       # listar ordenando según el tamaño
ls -F       # archivo*, directorio/
```

Comandos para el manejo de directorios

cd

La orden **cd** cambia el directorio de trabajo. Si no se especifica una ruta, cambiamos al directorio **/home/user**.

```
# cambiamos al directorio home del usuario
cd
# cambiamos a un directorio (ruta relativa)
cd carpeta1/
# no cambiamos de directorio
cd .
# cambiamos al directorio padre
cd ..
# cambiamos a un directorio oculto (del directorio
    actual)
cd .carpeta/
```

Comandos para el manejo de directorios

mkdir

La orden **mkdir** crea directorios, si aún no existen. Su uso es el siguiente:

```
mkdir [OPTION]... DIRECTORY...
```

Algunos parámetros de **mkdir** son:

- **-p** : No genera un error si el directorio ya existe. Además, crea los directorios padre necesarios si no existen, sin afectar los modos de archivo de los directorios existentes.
- **-m** : Establece el modo de archivo (permisos) para el nuevo directorio.

```
mkdir carpeta1 carpeta2 # creamos varios directorios
mkdir carpeta{3,4,5,6}  # creamos un directorio

# creamos un directorio que contiene otros directorios
mkdir -p carpeta4/subcarpeta/subsubcarpeta
# creamos un directorio con permisos específicos
mkdir -m 755
```

Comandos básicos para el manejo de directorios

rmdir

El comando **rmdir** elimina directorios si están vacíos. Su sintaxis es la siguiente:

```
rmdir [OPTION]... DIRECTORY...
```

Ejemplos:

```
# Eliminamos un directorio
rmdir carpeta5/
# Eliminamos un directorio y subdirectorios
rmdir -p carpeta4/subcarpeta/
# Podemos usar regex
rmdir carpeta*
```

Comandos para el manejo de archivos

Comandos para el manejo de archivos

touch (creación de un archivo)

Para crear un archivo podemos usar la orden **touch**. Su sintaxis general es la siguiente:

```
$ touch [OPCIÓN]... ARCHIVO...
```

Algunos parámetros para la orden **touch**:

- Crea un archivo: Si se proporciona un argumento **ARCHIVO** que no existe, se crea un archivo vacío (excepto si se especifica -h o -c).
- Cambia las marcas de tiempo (tiempo de acceso, tiempo de modificación, tiempo de cambio).
- El comando touch sin ninguna opción crea un nuevo archivo. Si el archivo existe, el comando actualizará el tiempo de acceso y de modificación a la hora actual sin cambiar su contenido.

Comandos para el manejo de archivos

touch (creación de un archivo)

Ejemplos de uso de la orden **touch**:

```
$ #creamos un archivo
$ touch archivo.txt
$ #creamos multiples archivos
$ touch archivo1.txt archivo2.txt
$ #creamos aún más archivo
$ touch archivo{3,4,5,6,7}.txt
$ ls
 archivo.txt   archivo2.txt  archivo4.txt  archivo6.txt
 archivo1.txt  archivo3.txt  archivo5.txt  archivo7.txt
```

Comandos para el manejo de archivos

Crear un archivo y agregarle contenido (con echo)

La orden **echo** muestra un texto en la terminal, es similar a un print de Python o un printf de C.

```
$ echo "muestra un texto en la terminal"
muestra un texto en la terminal
```

Podemos redirigir la salida de **echo** a un archivo con el operador ">".

```
# Agregamos contenido al archivo (sobrescribe el
    contenido)
$ echo "texto" > texto.txt
# Agregamos contenido al archivo (sin sobrescribir)
$ echo "más texto" >> texto.txt
$ cat texto.txt
texto
más texto
```

Comandos para el manejo de archivos

Crear un archivo y agregarle contenido (con cat)

Con el comando **cat**, es posible crear un archivo y agregarle contenido utilizando el operador ">" para redirigir el texto al archivo. Al escribir el comando "**cat > archivo1.txt**", se crea el archivo y puedes comenzar a añadir contenido. Para agregar varias líneas de texto, utiliza la tecla **Enter** al final de cada línea. Para finalizar y guardar el archivo, presiona **CTRL + D**.

```
1 $ cat > archivo.txt
2 texto
3 más texto
4 aún más texto
5 $ cat archivo.txt
6 texto
7 más texto
8 aún más texto
```

Comandos para el manejo de archivos

Ver el contenido de un archivo (con cat)

Para ver el contenido de un archivo, se usa la orden **cat**, se le da como argumento el nombre del archivo que deseamos consultar

```
# Vemos el contenido de un archivo llamado archivo.txt
cat archivo.txt
# Vemos el contenido de todos los archivos .txt
cat *.txt
# Vemos el contenido del archivo y el número de línea
cat -n archivo1.txt
# Vemos el conteido de todos los archivos del directorio
cat *
# Número de línea en solo las líneas con contenido
cat -b archivo.txt
```

Comandos para el manejo de archivos

file

La orden **file** se utiliza para determinar el tipo de archivo. En Linux, los nombres de archivo no están obligados a representar el contenido del archivo; se puede tener una archivo llamado archivo.png que en realidad no sea un PNG. El comando **file** analiza el contenido de los archivos y proporciona información sobre su tipo, en lugar de basarse únicamente en la extensión del archivo. La sintaxis general del comando es la siguiente:

```
file [OPCIÓN...] [ARCHIVO...]
```

Algunos parámetros de la orden file:

- **-p**, **–preserve-date** : Preserva los tiempos de acceso en los archivos.
- **-z**, **–uncompress** : Intenta examinar archivos comprimidos.

Ejemplos de uso:

```
# Determina el tipo de un archivo
$ file archivo.txt
# Determina el tipo de los archivos del directorio
    actual
$ file *
```

Comandos para el manejo de directorios y archivos

tree

El comando **tree** se utiliza para mostrar la estructura de directorios y archivos en forma de árbol. Su sintaxis general es la siguiente:

```
tree [OPCIONES] [DIRECTORIO...]
```

Si no se especifica un directorio, **tree** mostrará la estructura del directorio actual.

Algunos parámetros útiles e interesantes para este comando:

Opciones de listado

- **-a** : Muestra todos los archivos, incluidos los ocultos.
- **-d** : Lista solo los directorios.
- **-L** : Limita la profundidad de la visualización a un nivel específico.
- **-P** : Muestra solo los archivos que coinciden con el patrón dado.
- **-I** : No lista archivos que coinciden con el patrón dado.

Opciones de ordenamiento

- **-t** : Ordena los archivos por la fecha de última modificación.
- **-v** : Ordena los archivos alfanuméricamente por versión.
- **-r** : Invierte el orden de la lista.

tree

Opciones de información de archivos

- -s: Muestra el tamaño de cada archivo en bytes.
- -h: Muestra el tamaño de los archivos en un formato más legible (por ejemplo, KB, MB).
- -p: Muestra los permisos de cada archivo.
- -u: Muestra el propietario del archivo.
- -g: Muestra el grupo propietario del archivo.
- -l: Sigue enlaces simbólicos como si fueran directorios.

Opciones de información de salida

- -o filename: Redirige la salida a un archivo en lugar de a la consola.
- -X: Imprime una representación XML de la estructura del árbol.
- -J: Imprime una representación JSON de la estructura del árbol.

Copiar archivos o directorios

cp

El comando **cp** se utiliza para copiar archivos y directorios. Su sintaxis es la siguiente:

```
1 cp [OPCIÓN]... ORIGEN... DIRECTORIO
```

Algunos parámetros útiles:

- **-R**, **-r**, **–recursive** : Copia los directorios de manera recursiva.
- **-i**, **–interactive** : Pregunta antes de sobrescribir archivos existentes (si copias un archivo a un directorio que tiene el mismo nombre de archivo, el archivo se sobrescribirá con lo que estás copiando, con -i te preguntará antes de sobrescribir).
- **-v**, **–verbose** : Muestra información detallada sobre lo que se está copiando.

Podemos usar las banderas *****, **?**, **[]**.

Mover o renombrar archivos o directorios

mv

El comando **mv** se utiliza para mover o renombrar archivos. La sintaxis general del comando es la siguiente:

```
mv [OPCIÓN]... ORIGEN... DIRECTORIO
```

Algunos parámetros útiles:

- **-i**, **–interactive** : Solicita confirmación entes de sobrescribir archivos.
- **-f**, **–force** : No solicita confirmación antes de sobrescribir archivos.
- **-n**, **–no-clobber** : No sobrescribe archivos existentes.
- **-v**, **–verbose** : Muestra información detallada sobre lo que se está haciendo.

Ejemplos (renombra un archivo, mueve un archivo a un directorio, mueve y solicita confirmación antes de sobrescribir; respectivamente):

```
mv archivo_viejo.txt archivo_nuevo.txt
mv archivo1.txt archivo2.txt /ruta/al/directorio/
mv -i archivo.txt /ruta/al/directorio/
```

Borrar archivos o directorios

rm

El comando **rm** se utiliza para eliminar (desvincular) archivos y directorios. La sintaxix general del comando es:

```
1 rm [OPCIÓN]... [ARCHIVO]...
```

Algunos parámetros útiles:

- **-i** : Solicita confirmación antes de cada eliminación.
- **-I** : Solicita confirmación una vez antes de eliminar más de tres archivos o al eliminar recursivamente.
- **-r**, **-R**, **–recursive** : Elimina directorios y su contenido de manera recursiva.
- **-d**, **–dir** : Elimina directorios vacíos.

Los archivos no van a la "papelera" usando el comando rm, así que hay que tener cuidado.

Uso de la papelera

Para enviar un archivo a la papelera en vez de "eliminarlo definitivamente" como con rm, podemos usar:

```
gio trash [ARCHIVO/DIRECTORIO..]
```

Los archivos de la papelera están ubicados en la ruta:
~/.local/share/Trash/files/
Así que para visualizarlos:

```
ls .local/share/Trash/files/
```

Editores de texto

Editores de texto - VIM/NVIM

Introducción

Para editar un archivo desde la terminal, podemos usar VIM. Vim significa vi (mejorado), tal como su nombre lo indica, es una versión mejorada del clásico editor de texto vi. VIM es muy ligero y rápido. Además, está disponible en casi todas las distribuciones (para instalar o ya instalado). También existe una versión más nueva llamada neovim (nvim).
El uso de este editor es:

```
1 vim [argumento] [archivo ..]
```

No es necesario que el archivo esté creado.
En VIM existen diferentes modos. Los modos son "estados" en los que está el editor; cada modo está diseñado para tareas específicas.

- Normal: Modo pretedeterminado para navegar.
- Insert: Modo para introducir texto directamente.
- Command: Modo para ejecutar comandos.
- Visual: Modo para seleccionar texto.

Editores de texto - VIM/NVIM

Modo Normal - Navegación

En Vim no se usa el mouse, todo se hace con atajos de teclado. El modo normal es el modo predeterminado en el que Vim inicia. Mediante ciertas teclas podemos cambiar de modo.
En el modo **normal**, no podemos editar texto directamente, pero podemos visualizar el contenido del archivo y navegar por él.

- h, j, k, l: Mover el cursor a la izquierda, abajo, arriba y derecha, respectivamente.
- Con las teclas de flechas igual podemos mover el cursor.
- dd: Eliminar la línea actual.
- yy: Copiar la línea actual.
- x: Borrar caracteres.
- p: Pegar el texto seleccionado.

Editores de texto - VIM/NVIM

Modo Insert - Editar un archivo

En el modo **Normal** no podemos insertar texto, al ser el modo predeterminado en el que Vim inicia, puede resultar bastente confuso, especialmente si solo se desea abrir un archivo e ingresar texto. Para agregar contenido al archivo, necesitamos entrar en modo **Insert**, en este modo se puede escribir y editar textos como en un editor de texto convencional. Se puede entrar al modo insertar de las siguientes maneras:

- i - insertar texto.
- O - insertar texto en la línea anterior.
- o - insertar texto en la línea siguiente.
- a - añadir texto después del cursor.
- A - añadir texto al final de la línea.

Para regresar al modo **Normal**, se debe presionar la tecla Esc.

Editores de texto - VIM/NVIM

Modo Command - Guardado y salida en VIM

Para guardar y salir de Vim, debemos ingresar al modo **Command**; para ello se presiona **:** desde el modo **Normal**, a partir de ahí, podemos escribir los siguientes comandos.

- :w - escribe o guarda el archivo
- :q - salir de vim
- :wq - Escribir y salir

Editores de texto - VIM/NVIM

Modo Command - Busqueda

Para **buscar una expresión** simplemente escribe la tecla / y luego tu resultado de búsqueda. Por ejemplo: **/texto**. Una vez que presiones Enter, puedes usar **"n"** o **"N"** para para avanzar o retroceder en los resultados de búsqueda. Se puede usar expresiones regulares.
Con **?** se puede buscar en el archivo de texto hacia atrás.



Figure: Buscando "rounding" en vim

Editores de texto - VIM/NVIM

Modo Command - Reemplazar texto

Para buscar texto y reemplazarlo entramos en modo comando; se debe escribir lo siguiente:

```
:%s/patrón/nuevo_texto/g
```

- :% indica que la operación se aplicará a todo el archivo.
- s es el comando de sustitución.
- patrón es el texto que deseas buscar.
- nuevo_texto es el texto que deseas usar como reemplazo.
- g (opcional) significa que se reemplazarán todas las ocurrencias en cada línea. Si no lo incluyes, solo se reemplazará la primera ocurrencia de cada línea.

Editores de texto - nano

Navegación

Nano es un editor de archivos muy simple y fácil de usar. Muy accesible para aquellos que son nuevos en la línea de comandos. Para abrir un archivo en Nano, se usa el siguiente comando en la terminal:

```
1 nano [ARCHIVO..]
```

Una vez que estés en Nano, puedes navegar por el texto utilizando las teclas de flecha. También puedes usar las siguientes combinaciones:

- Ctrl + A: Ir al principio de la línea.
- Ctrl + E: Ir al final de la línea.
- Ctrl + Y: Desplazarse hacia arriba una página.
- Ctrl + V: Desplazarse hacia abajo una página.

Editores de texto - nano

Comandos

En Nano podemos ejecutar comandos con: **Ctrl + Tecla**

- **Ctrl + X** Salir
- **Ctrl + O** Escribir
- **Ctrl + K** Cortar la línea actual.
- **Ctrl + U** Pegar la línea cortada.
- **Ctrl + W** Buscar texto en el archivo.
- **Ctrl + C** Mostrar la posición del cursor (número de línea y columna).
- **Ctrl + G** Mostrar la ayuda de Nano.

Utilidades de la línea de comandos

Utilidades

less

La orden **less** muestra el contenido de un archivo o de la salida estándar en terminal. El texto se muestra de forma paginada, por lo que se puede navegar a través de un archivo de texto grande página por página. Less no tiene que leer todo el archivo de entrada antes de comenzar, por lo que con archivos de entrada grandes se inicia más rápido que los editores de texto como vim. Su sintaxis general de uso es:

```
$ less [ARCHIVO..]
# Puedes usar pipeline:
$ pacman -Ss gnome | less
```

less tiene una gran cantidad de parámetros, aquí algunos de ellos:

- Navega usando las teclas de fecha y las teclas de página.
- **q** - Se utiliza para salir de less y volver a tu shell.
- **g** - Se mueve al principio del archivo de texto.
- **G** - Se mueve al final del archivo de texto.
- **/búsqueda** - Puedes buscar texto específico dentro del archivo, antecede las palabras que deseas buscar con /
- **h** - Si necesitas un poco de ayuda sobre cómo usar less mientras estás en less, utiliza help.

Utilidades

sort

La orden **sort** se utiliza para ordenar las líneas de texto en archivos o desde la entrada estándar. La sintaxis general de **sort** es:

```
sort [OPCIÓN]... [ARCHIVO]...
```

Si no se especifica un archivo, **sort** leerá desde la entrada estándar. Algunas opciones útiles:

- **-b**, **–ignore-leading-blanks** : Ignora los espacios en blanco al principio de las líneas.
- **-f**, **–ignore-case** : Ignora las diferencias entre mayúsculas y minúsculas.
- **-g**, **–general-numeric-sort** : Ordena según el valor numérico general.
- **-h**, **–human-numeric-sort** : Ordena números legibles por humanos (e.g., 2K, 1G).
- **-n**, **–numeric-sort** : Ordena según el valor numérico de las cadenas.
- **-r**, **–reverse** : Invierte el resultado de las comparaciones.
- **-V**, **–version-sort** : Ordena números de versión dentro del texto.
- **-o**, **–output=FILE** : Escribe el resultado en el archivo especificado en lugar de la salida estándar.
- **-u**, **–unique** : Muestra solo la primera de las líneas con claves iguales.
- **-c**, **–check** : Verifica si la entrada está ordenada; no ordena.

Utilidades

uniq

El comando **uniq** se utiliza para filtrar líneas adyacentes coincidentes en un archivo o de la salida estándar. Es especialmente útil para eliminar duplicados en listas de texto. La sintaxis general del comando es:

```
uniq [OPCIÓN]... [ENTRADA [SALIDA]]
```

Si no se especifica un archivo de entrada, **uniq** leerá desde la estrada estándar. Si no se especifica un archivo de salida, escribirá en a salida estandar. Algunas opciones útiles:

- **-u**, **–unique** : Imprime solo líneas únicas.
- **-c**, **–count** : Prefija las líneas con el número de ocurrencias.
- **-d**, **–repeated** : Imprime solo las líneas duplicadas, una por cada grupo.
- **-D** : Imprime todas las líneas duplicadas.
- **-i**, **–ignore-case** : Ignora las diferencias de mayúsculas y minúsculas al comparar.

Utilidades

grep

La orden **grep** se utiliza para buscar patroes en archivos de texto o en la salida estándar. Es una herramienta poderosa para filtrar y encontrar información específica en grandes volúmenes de datos. La sintaxis general de esta orden es:

```
1 grep [OPCIÓN]... PATTERNS [ARCHIVO]...
```

Si no se especifica un archivo, **grep** leerá desde la entrada estándar. Podemos unsar regex en los patrones de búsqueda. Algunos parámetros útiles para **grep**:

- **-v** : Imprimir líneas que no contienen una palabra específica.
- **-i**, **–ignore-case** : Ignora las diferencias entre mayúsculas y minúsculas.
- **-w**, **–word-regexp** : Coincide solo con palabras completas.
- **-x**, **–line-regexp** : Coincide solo con líneas completas.

Utilidades

grep

Algunas opciones útiles para **grep**:

- -**B**, –**before-context=NUM** : Imprime NUM líneas de contexto antes de las coincidencias.
- -**A**, –**after-context=NUM** : Imprime NUM líneas de contexto después de las coincidencias.
- -**C**, –**context=NUM** : Imprime NUM líneas de contexto alrededor de las coincidencias.
- -**n**, –**line-number** : Imprime el número de línea con las líneas de salida.
- -**H**, –**with-filename** : Imprime el nombre del archivo con las líneas de salida.

awk..

Utilidades

figlet

Muestra carácteres grandes compuestos por carácteres normales de pantalla (ASCII art).

```
$ figlet hola
| |__   ___ | | __ _
| '_ \ / _ \| |/ _` |
| | | | (_) | | (_| |
|_| |_|\___/|_|\__,_|
```

Figlet puede imprimir una variedad de fuentes. Con el parámetro **-f** podemos especificar la fuente. Las fuentes disponibles son: banner, ivrit, shadow, smslant, big, lean, slant, standard, block, mini, small, term, bubble, mnemonic, smscript, digital, script, smshadow.
Con el parámetro **-f** también podemos especificar un archivo **.flf**

```
#Ejemplo con el parámetro -f
$ figlet -f mini hola
|_  _ | _.
| |(_)|(_|
```

Utilidades

fastfetch

Fastfetch es una herramienta para mostrar información del sistema de una forma visualmente atractiva. Imprime el logo de tu distribución en ASCII Art. Escrita principalmente en C, se centra en el rendimiento y la personalización. Este programa no viene preinstalado. El archivo de configuración se encuentra en la ruta "~/.config/fastfetch/config.jsonc".



Figure: Ejecución de fastfetch con uno de los archivos de configuración predeterminados

Utilidades

fastfetch

Algunos parámetros para **fastfetch**:

- **-l**, **–logo <logotipo>** : Establece el logotipo que se mostrará. Puede ser el nombre de un logotipo integrado o la ruta a un archivo de imagen. Utilice none para desactivar el logotipo.
- **–logo-type [TIPO]** : Establece el tipo de logotipo especificado con –logo. Los tipos disponibles incluyen auto, builtin, file, sixel, kitty y otros.
- **–logo-width <anchura>** : Establece la anchura del logotipo en caracteres (para logotipos de imagen).
- **–logo-height <altura>** : Establece la altura del logotipo en caracteres (para logotipos de imagen).
- **-c**, **–config <config>** : Utiliza el archivo de configuración o el ajuste preestablecido especificado. Especifica none para desactivar la carga de configuraciones adicionales.
- **–gen-config [RUTA]** Genera un archivo de configuración con las opciones especificadas en la línea de comandos. Si no se especifica ruta, el valor predeterminado es /.config/fastfetch/config.jsonc. Si la ruta es «-», la configuración se escribirá en stdout.
- **–file [RUTA]** : Usa un archivo para el logo. Puede ser una imágen si la terminal lo soporta (kitty, konsole, ghostty).

Utilidades

htop

Es un visor de procesos (como el administrador de tareas de Windows). Permite observar todos los procesos que se ejecutan en el sistema, junto con sus argumentos de línea de comandos, así como verlos en formato de árbol, seleccionar varios procesos y actuar sobre ellos a la vez. Podemos desplazarnos vertical y horizontalmente y seleccionar un proceso con el teclado o el mouse. Las tareas relacionadas con los procesos (eliminar, cambiar el nivel de prioridad) se pueden realizar sin introducir sus PID. No está instalado por defecto. Para iniciarlo, simplemente escribe en la terminal:

```
$ sudo pacman -S htop # Instalar
$ htop # Iniciar
```

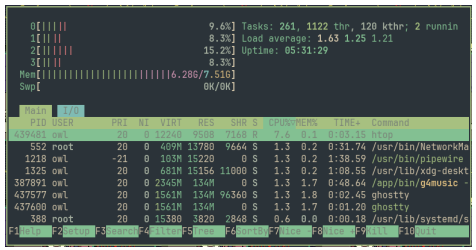


Figure: Ejecución de htop

Usuarios y grupos

Usuarios

Conceptos

Los usuarios y los grupos se utilizan en GNU/Linux para el control de acceso a directorios, archivos y periféricos del sistema. Linux ofrece mecanismos de control de acceso relativamente simples por defecto.

- Un **usuario** es cualquiera que use una computadora. En este caso, estamos describiendo nombres que representan a los usuarios. Puede ser un nombre real o un alias. Lo único que importa es que la computadora tenga un nombre para cada cuenta que cree, y es este nombre por que que una persona obtiene acceso a una computadora.
- Cualquier persona puede tener más de una cuenta, siempre que utilicen un nombre diferente para cada una de ellas. Hay algunos nombres reservados que no se pueden usar, como **root**.
- La administración de los usuarios se realiza con fines de seguridad al limitar el acceso de ciertas maneras específicas. El usuario **root** tiene acceso completo al sistema operativo y su configuración; está destinado solo para uso administrativo. Los usuarios sin privilegios pueden usar los programas **su** o **sudo** para la escalada de privilegios controlada.
- Los usuarios pueden pertenecer a un **grupo**, y del mismo modo, pueden añadirse a un grupo existente para utilizar el acceso privilegiado que dicho grupo concede.

Usuarios

Añadir un nuevo usuario

Para añadir un nuevo usuario, se utiliza el comando **useradd**:

```
#sintaxis general
$ useradd -m -G <grupo> -s <intérprete> <nombre>
```

- **-m** : Crea el directorio de usuario (/home/user). Dentro de este directorio, un usuario que no es superusuario puede escribir archivos, eliminarlos, instalar programas, etc.
- **-G** : Introduce una lista de grupos suplementarios de los que el usuario también es miembro. Cada grupo está separado del siguiente mediante una coma, sin espacios intermedios. Por defecto el usuario pertenece solo al grupo inicial.
- **-s** : Define la ruta y el nombre de archivo del intérprete de línea de órdenes de inicio de sesión predeterminado del usuario. Una vez que se completa el arranque, el intérprete de línea de órdenes de inicio de sesión predeterminado es el que se especifica aquí. Asegúrese de que el intérprete de línea de órdenes elegido esté instalado si es distinto a Bash.

Usuarios

whoaomi - Establecer contraseña a un usuario

Para saber el nombre del usuario actual, usamos el comando whoami.

```
whoami
```

La contraseña debe definirse después de crear al usuario, utilizando el comando **passwd**. Aunque no es necesario proteger al usuario con una contraseña, se recomienda encarecidamente hacerlo. Ejemplo creando un usuario y estableciendole una contraseña:

```
#Creamos un usuario llamado "archie"
$ sudo useradd -m archie
#Establecemos una contraseña a "archie"
$ sudo passwd archie
```

Si el usuario ya tenía una contraseña, con **passwd** podemos cambiarla, siempre y cuando recordemos la contraseña ya existente.

Administración de usuarios

Cambiar el nombre de inicio de sesión de un usuario

```
$ usermod -l [nombrenuevo] [nombreantiguo]
```

Borrar un usuario. Las cuentas de usuario se pueden eliminar con la orden **userdel**.

```
$ userdel -r [usuario]
```

Cambiar el directorio personal de un usuario. Para cambiar el directorio de inicio de un usuario:

```
$ usermod -d /mi/nuevo/directorio/personal -m usuario
```

La opción -m también crea automáticamente el nuevo directorio y mueve el contenido allí.

Usuarios

root, superusuario

En sistemas operativos de tipo Unix, el **superusuario** o **root** es el nombre convencional de la cuenta de usuario que posee todos los derechos en todos los modos (monousuario o multiusuario). Normalmente es la cuenta de administrador. El usuario root puede llevar a cabo las tareas que un usuario común no podría, tales como cambiar el propietarios o los permisos de un archivo. No es recomendable utilizar el superusuario root para una simple sesión de uso habitual, ya que esto puede poner en riesgo el sistema al otorgar acceso privilegiado a cada programa en ejecución. En estos casos, es preferible utilizar una cuenta de usuario no privilegiado y utilizar el comando su o sudo para acceder a los privilegios de root según sea necesario.

Usuarios

sudo

Sudo permite al administrador del sistema delegar autoridad para otorgar a ciertos usuarios (o grupos de usuarios) la capacidad de ejecutar órdenes como superusuario (root). A diferencia de su, que lanza un shell de superusuario que permite que todas las demás órdenes tengan acceso de superusuario, sudo ofrece una elevación temporal de privilegios a una sola orden. Sudo registra todas las órdenes y los intentos de acceso fallidos para la auditoría de seguridad.

```
1 $ pacman -S sudo #Instalamos sudo
```

El archivo **sudoers** es un archivo de configuración que define los permisos y las reglas para el uso de **sudo**. Este archivo especifica qué usuarios o grupos pueden ejecutar comandos como superusuario y bajo qué condiciones. Generalmente se encuentra en */etc/sudoers*.

Usuarios

sudoers

Podemos editar el archivo sudoers para que un usuario puede usar sudo. Si bien podemos hacer esta tarea con nano, nvim o cualquier otro editor de texto "así sin más", se recomienda encarecidamente el uso de visudo. El archivo sudoers no debe tener errores de sintaxis, cualquier error hará sudo inutilizable. Visudo bloquea el archivo sudoers, guarda los cambios en un archivo temporal y comprueba los errores sintácticos antes de copiarlo en */etc/sudoers*.

El editor predeterminado para visudo es vi. Para usar un editor diferente una vez, simplemente configuramos una variable antes de llamar a visudo, de la siguiente manera:

```
1 #Configuramos nano
2 $ EDITOR=nano visudo
3 #Configuramos neovim
4 $ EDITOR=nvim visudo
```

Usuarios

sudo, grupos

Una vez en visudo nos movemos aproximadamente a la línea 120 y veremos algo como lo siguiete:

```
1 root  ALL=(ALL:ALL) ALL
```

Entonces podemos agregar el usuario deseado, escribiendo lo mismo que en root

```
1 root                  ALL=(ALL:ALL) ALL
2 NOMBRE_DE_USUARIO     ALL=(ALL:ALL) ALL
```

Grupos

Los usuarios pueden agruparse en un **grupo**, y del mismo modo, pueden añadirse a un grupo existente para utilizar al acceso privilegiado que dicho grupo concede. Algunos grupos del sistema (hay muchos más) son:

- **wheel** : Grupo de administración, comúnmente utilizado para dar privilegios para realizar acciones administrativas. Tiene acceso de lectura total a los archivos journal y el derecho de administrar impresoras en CUPS. Puede ser utilizado también para dar acceso a las utilidades sudo y su (ninguno lo usa por defecto).
- **http** : Acceso a archivos servidos por servidores HTTP.
- **root** : Administración y control completo del sistema (root, admin).
- **storage** : Utilizado para obtener acceso a unidades extraíbles como discos duros USB, unidades flash, reproductores de MP3; permite al usuario montar dispositivos de almacenamiento.

Notese la importancia de root y del grupo wheel. Para permitir que los miembros del grupo wheel tengan acceso a sudo, editamos el archivo sudoers, descomentando la siguiente línea:

```
1 %wheel        ALL=(ALL:ALL) ALL
```

Grupos

Administración de grupos

/etc/group es el archivo que define los grupos del sistema.
También está su compañero gshadow que rara vez se utiliza. El archivo */etc/group* se utiliza para almacenar información general sobre los grupos de usuarios, mientras que */etc/gshadow* se centra en la seguridad y la gestión de contraseñas de grupo.
La orden groups muestra la pertenencia al grupo:

```
1 $ groups usuario
```

Si se omite usuario, se muestran los grupos a los que pertenece el usuario actual.
Para listar todos los grupos en el sistema:

```
1 $ cat /etc/group
```

Añada un usuario a un grupo con la orden gpasswd

```
1 $ gpasswd -a <usuario> <grupo>
```

Gestión de grupos

Agregar usuarios a un grupo, eliminar usuarios a un grupo

Alternativamente, añada un usuario a grupos adicionales con usermod (reemplace grupos_adicionales por una lista separada por comas):

```
$ usermod -aG <grupos_adicionales> <usuario>
```

Para eliminar un usuario de un grupo.

```
$ gpasswd -d <usuario> <grupo>
```

Permisos

Permisos

Los sistemas de archivos utilizan permisos para regular el nivel de interacción que los procesos del sistema pueden tener con archivos o directorios. Utilizamos la opción *-l* de la orden **ls** para ver los permisos, por ejemplo:

```
$ ls -l /ruta/al/directorio
total 128
drwxr-xr-x 2 archie archie  4096 Jul  5 21:03 Escritorio
drwxr-xr-x 6 archie archie  4096 Jul  5 17:37 Documentos
drwxr-xr-x 2 archie archie  4096 Jul  5 13:45 Descargas
-rw-rw-r-- 1 archie archie  5120 Jun 27 08:28 clientes.
    ods
-rw-r--r-- 1 archie archie  3339 Jun 27 08:28 todo
-rwxr-xr-x 1 archie archie  2048 Jul  6 12:56 miscript.
    sh
```

La primera columna es en lo que debemos enfocarnos. Por, ejemplo: *drwxr-xr-x*. Los permisos de los sistemas tipo UNIX se dividen en tres clases, conocidas como usuario (los permisos que tiene el propietario sobre el archivo), grupo (los permisos que tiene el grupo sobre el archivo) y otros (los permisos que tienen todos los demás usuarios sobre el archivo).

Permisos

Una forma de representar los permisos es mediante la notación simbólica. Este esquema permite representar permisos en una serie de 10 caracteres (ejemplo: drwxr-xr-x).

- El primer carácter indica el tipo de archivo.
  - - denota un archivo regular
  - **d** denota un directorio
  - **b** denota un archivo especial de bloques
  - **c** denota un archivo especial de caracteres
  - **l** denota un enlace simbólico
  - **p** denota un tubo nombrado
  - **s** denota un socket de dominio
- Tres grupos de tres caracteres
  - Primero: lo que el grupo o propietario puede hacer.
  - Segundo: lo que los miembros del grupo de usuarios pueden hacer.
  - Tercero: lo que el resto de usuarios pueden hacer
- La tupla
  - **r** si el bit de lectura está asignado, '-' en caso contrario.
  - **w** si el bit de escritura está asignado, '-' en caso contrario.
  - **x** si el bit de ejecución está asignado, '-' en caso contrario.

Permisos

Cambiar los permisos, chmod

Para cambiar los permisos de un archivo, se utiliza la orden **chmod**. Podemos usar números para editar los permisos para propietarios, grupos y otros al mismo tiempo. La sintaxis básica del comando es la siguiente:

```
$ chmod xxx archivo
```

xxx es un número de 3 dígitos donde cada dígito puede ser cualquier valor del 0 al 7. El primer dígito se aplica a los permisos para el propietario, el segundo dígito se aplica a los permisos para el grupo y el tercer dígito se aplica a los permisos para todos los demás. En esta notación numérica, los valores **r**, **w** y **x** tienen su propio valor numérico:

- **r**=4
- **w**=2
- **x**=1

Para obtener un número de 3 dígitos, debe considerar qué permisos desea que tengan el propietario, el grupo y todos los demás, y luego totalizar sus valores.

Permisos

Cambiar los permisos, chmod

Por ejemplo, si desea otorgar al propietario de un directorio permisos de lectura, escritura y ejecución, y desea que el grupo y todos los demás solo tengan permisos de lectura y ejecución, obtendría los valores numéricos de la siguiente manera:

- **Propietario**: rwx=4+2+1=7
- **Grupo**: r-x=4+0+1=5
- **Otros**: r-x=4+0+1=5

Entonces usamos el comando

```
$ chmod 755 archivo
```

Para ver los permisos existentes en formato numérico, se utiliza la orden stat.

```
$ stat -c %a archivo
```

Donde la opción %a especifica la salida en formato numérico.

reboot, shutdown

Control del sistema

Apagado y reinicio

Se pueden usar las siguientes órdenes para reiniciar la computadora:

```
$ reboot
```

o bien:

```
$ shutdown -r
```

Se pueden usar las siguientes órdenes para apagar la computadora (ahora mismo)

```
$ poweroff
```

o bien:

```
$ shutdown now
```

Si se desea apagar la computadora a una hora en específico:

```
shutdown -h [hora xx:xx]
```

Por ejemplo:

```
$ shutdown -h shutdown -h 23:59
Shutdown scheduled for Sun 2025-09-07 23:59:00 CST, use
    'shutdown -c' to cancel.
```

Configuración de red

Ping

En esta sección describiremos como configurar conexiones de red. La orden **ping** se utiliza para comprobar si se puede comunicarse con un host.

```
$ ping www.example.com
PING www.example.com (93.184.216.34): 56(84) data bytes
64 bytes from 93.184.216.34: icmp_seq=0 ttl=56 time
    =11.632 ms
64 bytes from 93.184.216.34: icmp_seq=1 ttl=56 time
    =11.726 ms
64 bytes from 93.184.216.34: icmp_seq=2 ttl=56 time
    =10.683 ms
...
```

Por cada respuesta recibida, la utilidad ping imprimirá una línea como la anterior hasta que la interrumpa (mediante Ctrl+c) de forma interactiva. Si se recibe un mensaje de error o no recibe respuesta, esto puede estar relacionado con una configuración incompleta, pero también con su puerta de enlace predeterminada o su proveedor de servicios de Internet (ISP).

Interfaces de red

Una interfaz de red es una parte de un dispositivo que permite la conexión y la comunicación con otros dispositivos a través e una red. Tanto los nombres de las interfaces tanto las cableadas como las inalámbricas se pueden encontrar a través de alguna de las siguientes órdenes:

```
$ ip link
$ ls /sys/class/net
$ cat /proc/net/dev
```

Es importante destacar algunas diferencias entre los comandos anteriores:

- **ip link** : Resumen de las interfaces de red disponibles y su estado.
- **ls /sys/class/net** : Lista las interfaces de red disponibles
- **cat /proc/net/dev** : Proporciona estadísticas detalladas sobre el tráfico de red de cada interfaz.

Activar y desactivar interfaces de red

Las interfaces de red se pueden activar y desactivar utilizando la orden **ip link set interfaz up|down**. También se puede usar **ip link** para comprobar el estado de una interfaz de red. Por ejemplo, imaginemos que deseamos comprobar el estado de la interfaz **wlp2s0**, y en caso de estar desactivada, activarla.

```
1 $ ip link show dev wlp2s0
2 3: wlp2s0: <BROADCAST,MULTICAST,DOWN> mtu 1500 qdisc
      pfifo_fast state DOWN group default qlen 1000
3     link/ether 00:1A:2B:3C:4D:5E brd ff:ff:ff:ff:ff:ff
4 $ ip link set up wlp2s0
5 3: wlp2s0: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc
      pfifo_fast state UP group default qlen 1000
6     link/ether 00:1A:2B:3C:4D:5E brd ff:ff:ff:ff:ff:ff
```

Podemos ver en la salida del primer comando *state DOWN*, lo que indica que la interfaz está desactivada, después de activarla, podemos leer *state UP* que indica que la interfaz está activada.

Direcciones IP

Las direcciones IP se gestionan utilizando **ip-address**. Para listar las direcciones IP usamos la orden:

```
$ ip address show
```

Para **añadir una dirección IP** a una interfaz:

```
# ip address add dirección/longitud_prefijo broadcast +
    dev interfaz
```

Por ejemplo:

```
$ sudo ip address add 192.168.1.74/24 dev wlp2s0
```

Para **eliminar una dirección IP** de una interfaz:

```
# ip address del dirección/longitud_prefijo dev interfaz
```

Por ejemplo:

```
$ sudo ip address del 192.168.1.100/24 dev wlp2s0
```

Gestor de redes

Un gestor de redes le permite administrar la configuración de la conexión de la red. Existen muchas herramientas para elegir, pero todas son mutuamente excluyentes; no debe ejecutar dos denominos simultáneamente. Aquí algunos gestores de red:

- ConnMan
- netctl
- NetworkManager
- systemd-networkd

En este caso nos centraremos en NetworkManager. La herramienta de línea de comandos para controlar la red con NetworkManager es **nmcli**.

nmcli

Listar todas las conexiones configuradas:

```
$ nmcli connection show
```

Para listar todas las redes Wi-Fi disponibles:

```
$ nmcli device wifi list
```

Para conectarte a una red Wi-Fi:

```
$ nmcli device wifi connect "nombre_red" password "
    tu_contraseña"
```

Para desconectarte de una red Wi-Fi:

```
$ nmcli device disconnect nombre_dispositivo
```

En el directorios **/etc/NetworkManager/system-connections** se almacena información sobre nuestras redes.

SSH

Secure Shell o SSH es un protocolo de red que permite el intercambio de datos sobre un canal seguro entre dos computadoras. SSH usa criptografía de clave pública para autenticar al equipo remoto y permitir al mismo tiempo autenticar el equipo remoto y permitir al mismo autenticar al usuario si es necesario. SSH se suele utilizar para iniciar una sesión en una máquina remota, donde poder ejecutar órdenes, pero también permite la tunelización, el reenvío de puertos TCP de forma arbitraria y de conexiones X11. Un servidor SSH, por defecto, escucha el puerto TCP 22. Un programa cliente de SSH es utilizado, generalmente, para establecer conexiones a un demonio sshd que acepta conexiones remotas. OpenSSH (OpenBSD Secure Shell) es un conjunto de programas de computadora que proveen una sesión de comunicación encriptada en una red informática que utiliza el protocolo SSH, lo instalamos desde los repositorios oficiales:

```
1 $ sudo pacman -S openssh
```

SSH

Para activar el demonio:

```
$ sudo systemctl enable sshd.service
```

Algunas órdenes de ssh:

Para conectarse a un servidor se usa la siguiente orden (si no se especifica un puerto *-p [puerto]*, por defecto se usa el 22):

```
ssh -p [puerto] [usuario]@[dirección-servidor]
```

También podemos ejecutar un comando y salir de ssh de la siguiente manera:

```
ssh -p [puerto] [usuario]@[dirección-servidor] [command]
```

SSH

Descargar y subir archivos a un servidor ssh

Para descargar y subir un archivo con SSH, se puede emplear la herramienta **scp**. Para subir un archivo desde la máquina local a un servidor, se usa el siguiente comando:

```
$ scp /ruta/local/archivo.txt usuario@host:/ruta/remota/
```

Para descargar un archivo desde el servidor a la máquina local, se usa el siguiente comando:

```
$ scp usuario@host:/ruta/remota/archivo.txt /ruta/local/
```

SSH

Configurar ssh con clave pública y privada

Por defecto el servidor ssh intentará autenticar con contraseña, permitiendo así que un usuario malicioso intente ganar acceso con un ataque de fuerza bruta en la contraseña. Uno de los métodos más efectivos para para proteger al sistema contra ese tipo de ataques es desactivando el inicio de sesión con contraseña complamente. En la máquina local, genera un par de claves ssh, con tipo de clave RSA y especificando el tamaño de la clave (recomendado 4006 bits):

```
$ ssh-keygen -t rsa -b 4096
```

La ubicación predeterminada es */.ssh/id_rsa*.
Una vez que se generaron el par de claves, se necesita copiar la clave al servidor

```
$ ssh-copy-id usuario@host
```

En el servidor, se edita el archivo de configuración */.ssh/sshd_config*, debe tener esta línea:

```
$ PasswordAuthentication no
```

Debemos tener los permisos correspondientes:

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/authorized_keys
```

Descargar archivos

Las herramientas cURL y wget son muy útiles para descargar archivos y realizar transferencias de datos a través de la red.

Para realizar una descarga en un archivo en específico:

```
$ curl --output filename URL
```

Para descargar un archivo con wget se usa la siguiente orden:

```
$ wget <url>
```

Instalación de programas y paquetes

Instalación de programas

Flatpak

Flatpak es un sistema de compilación, distribución y ejecución de aplicaciones de escritorio aisladas en Linux. Puedes instalar aplicaciones Flatpak de la misma manera, sin importar la distribución de GNU/Linux que estés utilizando.

- Flatpak permite instalar apps universales y aisladas del sistema.
- Flathub es el principal repositorio de Flatpak.
- Compatible con interfaces gráficas (Gnome Software, KDE Discover).

Para instalar flatpak en Arch o derivadas:

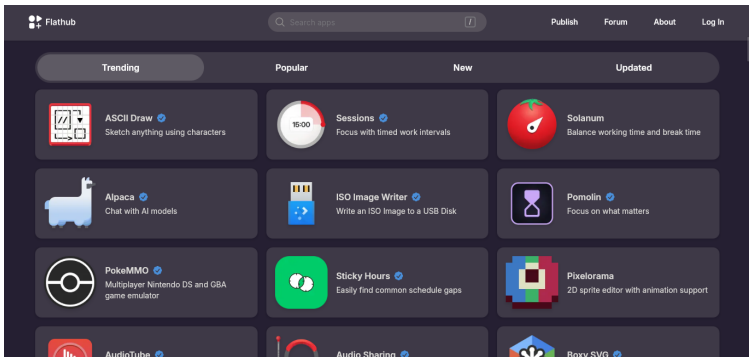
```
1 $ sudo pacman -S flatpak
```

Instalación de programas

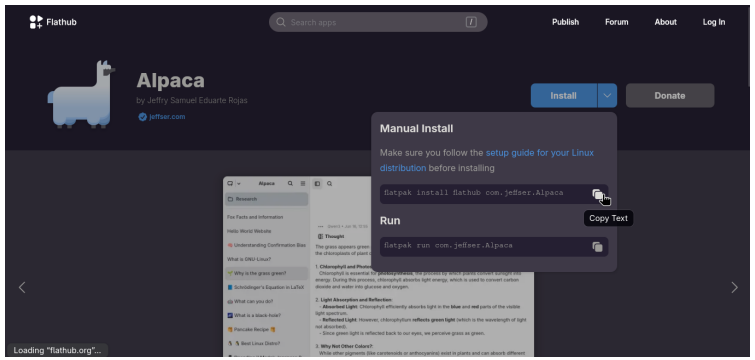
Flathub

El sitio flathub https://flathub.org/ (link aquí) es un excelente lugar para buscar aplicaciones flatpak. Para descargar aplicaciones de flatpak y flathub, debemos añadir el repositorio:

```
# Agregar Flathub
$ flatpak remote-add --if-not-exists flathub https://
    flathub.org/repo/flathub.flatpakrepo
```



(a) Buscando aplicaciones en Flathub



(b) Solo pega esto en la terminal para instalar la aplicación

Instalación de programas

Instalar, borrar y listar aplicaciones flatpak

Para instalar, eliminar, listar, ejecutar una aplicación y actualizarlas, usan las siguientes órdenes:

```
# Agregar un repositorio:
$ flatpak remote-add [opciones] <nombre_repositorio>  <
    url_repositorio>
# Instalar aplicación
$ flatpak install <nombre_repositorio>  <
    nombre_aplicación>
# Listar aplicaciones instaladas
$ flatpak list
# Eliminar una aplicación
$ flatpak uninstall <nombre_aplicación>
# Ejecutar una aplicación:
flatpak run <nombre>
# Actualizar flapak
$ flatpak update
```

Instalación de programas

Instalar, borrar y listar aplicaciones flatpak

Aquí un ejemplo instalando Spotify

```
# Instalar aplicación
$ flatpak install flathub com.spotify.Client
# Listar aplicaciones instaladas
$ flatpak list
# Eliminar aplicación
$ flatpak uninstall com.spotify.Client
```

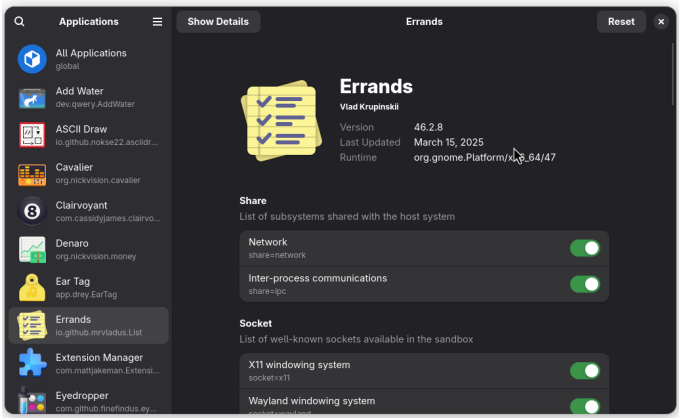
También es posible instalar aplicaciones flatpak con herramientas gráficas, como **Warehouse** o, si se usa GNOME como entorno de escritorio, desde **Gnome Software**; son similares a una "tienda de aplicaciones" como AppStore o PlayStore.

```
# Instalar Warehouse
$ flatpak install flathub io.github.flattool.Warehouse
```

Instalación de programas

Gestión de permisos de flatpak

Los permisos de Flatpak son configuraciones que determinan qué recursos del sistema puede acceder una aplicación instalada a través de Flatpak. Estos permisos controlan el acceso a diferentes aspectos del entorno del sistema, como archivos, dispositivos, redes y servicios. Es útil configurarlos por razones de seguridad, privacidad, prevención de errores o solo por personalización. Existe una herramienta gráfica llamada Flatseal (link aquí) con la cual podemos administrar los permisos de flatpak.



Instalación de programas, gestión de paquetes

Pacman

El gestor de paquetes **pacman** es una de las principales características distintivas de Arch Linux (y derivadas). Combina un simple formato de paquetes binarios con un fácil sistema de compilación de paquetes el objetivo de pacman es hacer posible gestionar fácilmente los paquetes, tanto si son los de los repositorios oficiales de Arch como las compilaciones realizadas por los propios usuarios.

Instalación de programas, gestión de paquetes

Pacman - Instalación

Para instalar paquetes, usamos las siguientes órdenes (esto también puede instalar un grupo de paquetes):

```
pacman -S nombre_del_paquete1 nombre_del_paquete2 ...
```

Para instalar un número de paquetes que comparten patrones similares en sus nombres uno puede usar la expansión de llaves. Por ejemplo:

```
pacman -S plasma-{desktop,mediacenter,nm}
```

Esto se puede ampliar a los niveles que sea necesario:

```
pacman -S plasma-{workspace{,-wallpapers},pa}
```

Para instalar una lista de paquetes con una expresión regular:

```
pacman -S $(pacman -Ssq expresion-regular_paquete)
```

Se puede usar la opción *–noconfirm* para que pacman no pida confirmación.

Instalación de programas, gestión de paquetes

Pacman - Borrar paquetes

Para desintalar un paquete dejando todas sus dependencias instaladas:

```
pacman -R nombre_del_paquete
```

Para eliminar un paquete y sus dependencias (siempre que no sean usadas por ningún otro paquete instalado):

```
pacman -Rs nombre_del_paquete
```

Para eliminar un paquete, sus dependencias y todos los paquetes que dependen de esas dependencias:

```
pacman -Rsc nombre_del_paquete
```

La orden de arriba es recursiva, y debe utilizarse con precaución, ya que puede eliminar muchos paquetes potencialmente necesarios.

Instalación de programas, gestión de paquetes

Pacman - Buscar paquetes

Pacman puede consultar la base de datos de los paquetes presentes en el sistema con la opción -Q, las bases de datos de los servidores remotos con la opción -S y los archivos presentes en dichas bases con la opión -F.
Pacman puede buscar paquetes en la base de datos, la búsqueda se realiza tanto por los nombres como por las descripciones de los paquetes:

```
pacman -Ss cadena1 cadena2 ...
```

Para buscar paquetes ya instalados:

```
pacman -Qs cadena1 cadena2 ...
```

Para buscar nombres de archivos de paquetes en los paquetes de los servidores remotos:

```
pacman -F cadena1 cadena2 ...
```

Para mostrar información detallada de un paquete, se usa la opción *i*, sin importar si se realiza una busqueda en los paquetes ya instalados o en los servidores:

```
pacman -Si nombre_del_paquete
```

Instalación de programas, gestión de paquetes

Pacman - Actualizar

Pacman puede actualizar todos los paquetes del sistema con una sola orden. Esto proceso puede durar bastante dependiendo de cuánto tiempo haya estado el sistema sin actualizar. La siguiente orden sincroniza las bases de datos de los repositorios y actualiza los paquetes del sistema (excluyendo los paquetes «locales» que no estén en los repositorios configurados):

```
1 pacman -Syu
```

- **-S** : Instalar o actualizar paquetes.
- **-y** : Sincronizar la base de datos de los repositorios.
- **-u** : Actualizar todos los paquetes instalados a sus versiones más recientes.

Instalación de programas, gestión de paquetes

Pacman - Actualizar

Sincroniza repositorios y actualiza los paquetes instalados a las versiones más recientes.

```
pacman -Syu
```

Sincroniza forzadamente los repositorios y permite downgrade de paquetes si es necesario.

```
pacman -Syyuu
```

Solo sincroniza la base de datos de los repositorios, sin actualizar los paquetes instalados.

```
pacman -Sy
```

Instalación de programas, gestión de paquetes

Pacman - Configuración de mirrors

El archivo **/etc/pacman.d/mirrorlist** contiene una lista de mirrors, define qué servidores usar para descargar los paquetes, pacman los usa en orden, si el primer mirror falla o es lento, va al siguiente. Es recomendable editar este archivo para tener mirrors rápidos y así, reducir el tiempo de actualización o instalación de paquetes. **Reflector** es un script escrito en Python que puede recuperar la última lista de servidores de réplicas, permite filtrar la mayoría de los servidores de réplicas actualizados, ordénarlos por velocidad y sobrescribir el archivo **/etc/pacman.d/mirrorlist**.
Para instalar reflector:

```
1 pacman -S reflector
```

Instalación de programas, gestión de paquetes

Pacman - Configuración de mirrors

Aquí unos ejemplos de uso de reflector:
Para ver todas las órdenes disponibles, ejecuta la siguiente órden:

```
$ reflector --help
```

Para ver los 5 mirrors más rápidos (sin sobrescribir el archivo */etc/pacman.d/mirrorlist*):

```
$ reflector --latest 5 --sort rate
```

Nota: a partir de aquí, los siguientes ejemplos sobrescriben el archivo */etc/pacman.d/mirrorlist*, por lo que se recomienda hacer una copia de seguridad.

```
$ sudo cp /etc/pacman.d/mirrorlist /etc/pacman.d/
    mirrorlist.backup
```

Instalación de programas, gestión de paquetes

Pacman - Configuración de mirrors

Busca los 10 mirrors más recientes, los ordena por velocidad y sobrescribe el archivo de mirrors existente:

```
$ sudo reflector --latest 10 --sort rate --save /etc/
    pacman.d/mirrorlist
```

Para seleccionar los 100 servidores de réplicas HTTPS más recientemente sincronizados, ordenarlos por velocidad de descarga y sobrescribir el archivo /etc/pacman.d/mirrorlist:

```
$ reflector --latest 100 --protocol https --sort rate --
    save /etc/pacman.d/mirrorlist
```

También se pueden especificar mirrors de un país:

```
$ sudo reflector --country Spain --latest 5 --sort rate
    --save /etc/pacman.d/mirrorlist
```

Instalación de programas, gestión de paquetes

Pacman - Configuración de mirrors

Puedes automatizar reflector con un timer de systemd para que actualice tus mirrors cada cierto tiempo:

```
$ sudo systemctl enable reflector.timer
$ sudo systemctl start reflector.timer
```

El archivo de configuración de reflector:

```
$ sudo nvim /etc/xdg/reflector/reflector.conf
```

Allí puedes poner tu configuración preferida, por ejemplo:

```
--country Mexico,United_States,Japan,Spain
--latest 10
--sort rate
--save /etc/pacman.d/mirrorlist
```

Instalación de programas, gestión de paquetes

AUR

Arch User Repository (AUR) es un repositorio promovido por los usuarios de la comunidad de Arch. Este contiene descripciones de los paquetes (PKGBUILD) que le permiten compilar un paquete desde el código fuente con makepkg y luego instalarlo a través de pacman. AUR fue creado para organizar y compartir paquetes nuevos de la comunidad y ayudar a acelerar la inclusión de los paquetes más populares en el repositorio extra. Un buen número de paquetes nuevos que entran en los repositorios oficiales tienen su origen en AUR. En AUR, los usuarios pueden aportar sus propias compilaciones de paquetes (PKGBUILD y los archivos relacionados). Desde el siguiente sitio se pueden buscar programas AUR: (AUR

Instalación manual:

```
$ git clone https://aur.archlinux.org/paquete.git
$ cd paquete
$ makepkg -si
```

Solo opcionalmente, se recomienda (por orden) clonar en el diretorio /tmp.

Puedes crear un directorio temporar con **mktemp -d**.

Instalación de programas, gestión de paquetes

AUR Helper: paru

¿Qué es un AUR Helper? Son herramientas que facilitan la interacción con el **AUR**, automatizando tareas como la búsqueda, descarga, compilación e instalación de paquetes desde dicho repositorio.
Paru es un AUR helper moderno escrito en Rust. Permite instalar paquetes tanto del repositorio oficial como del AUR usando un solo comando, con una interfaz cómoda e interactiva. También permite la actualización y limpieza de paquetes AUR.

Instalación de paru (manual):

```
$ git clone https://aur.archlinux.org/paru.git
$ cd paru
$ makepkg -si
```

Uso básico de paru:

- `paru -Ss nombre` — Buscar un paquete.
- `paru -S nombre` — Instalar un paquete (oficial o AUR).
- `paru -R nombre` — Eliminar un paquete.
- `paru -Syu` — Actualizar todos los paquetes del sistema, incluyendo AUR.

Al igual que otros AUR helpers, `paru` gestiona dependencias automáticamente

Stdout, stdin, stderr, pipeline

Redirección y Tuberías

- stdout: salida estándar (>)
- stdin: entrada estándar (<)
- stderr: errores (2>)
- pipeline: conectar salida de un comando a otro con |

Ejemplos:

```
# Guardar salida en archivo
$ ls > archivos.txt

# Leer desde archivo
$ wc -l < archivos.txt

# Guardar errores
$ comando 2> error.log

# Usar pipeline
$ cat archivo.txt | grep error | sort | uniq
```

Gestión de procesos y demonios

- Ver procesos: `ps`, `top`, `htop`
- Enviar señales: `kill`, `killall`
- Administrar servicios: `systemctl`

Ejemplos:

```
# Ver procesos activos
$ ps aux

# Matar proceso
$ kill PID

# Ver estado de servicio
$ systemctl status sshd

# Iniciar y habilitar demonio
$ sudo systemctl start sshd
$ sudo systemctl enable sshd
```

Cron y tareas programadas

- Permite ejecutar comandos en horarios específicos.
- Cada usuario tiene su propio `crontab`.

Ejemplo:

```
# Editar tareas programadas
$ crontab -e

# Ejecutar un script cada día a las 2AM
0 2 * * * /home/usuario/scripts/respaldo.sh
```

Shells e intérpretes

- Bash es el más común, pero existen zsh, fish, etc.
- Archivos de configuración:
  - `.bashrc`, `.zshrc`, `.config/fish/config.fish`
- Puedes cambiar de shell con `chsh`.

Ejemplo:

```
# Cambiar a zsh
$ chsh -s /bin/zsh

# Crear alias
alias actualizar='sudo pacman -Syu'
```

Bash scripting básico

- Un bash script es un archivo con comandos de shell.
- Se ejecuta como programa si tiene permisos.

Ejemplo:

```
#!/bin/bash
echo "Hola, $USER"
date
```

Ejecutar:

```
$ chmod +x script.sh
$ ./script.sh
```